EMADS

Architecture, code and interview guide

30 source-linked answers, executed test evidence, statistical caveats and interview practice.

Review scope	Verified provenance
Analysis date	2026-11-02 (session-provided date)
Inspected commit	292d600919934449e4ae781afe05662b80b07704
Working tree	Not clean on arrival: runtime CSV/model/plot/log files already modified. Application source had no reported tracked changes. This review does not reset or overwrite those existing artifacts.
Method	Complete application source and tests inspected; offline test suite executed; targeted probes; nine historical PDFs text-inspected; final guide rendered and visually reviewed.
Verification boundary	No live external API calls, dependency installation or project source modifications. No CV or separate thesis report supplied.
Existing tests	23 passed; 0 failures; 0 errors; 0 skipped. Offline isolated review harness.
Reading guide	Implemented = code path exists. Partial = useful behavior with a material gap. Absent = no implementation found. Proposed = future work, not performed.

Privacy: credentials and .env values are excluded. Examples are synthetic; private dataset rows and historical run metric values are not reproduced.

Abbreviations used throughout: API = application programming interface; SDK = software development kit; CV = cross-validation (or curriculum vitae when explicitly discussing your CV); EDA = exploratory data analysis; SHAP = SHapley Additive exPlanations; TPE = Tree-structured Parzen Estimator; RMSE = root mean squared error; R2 = coefficient of determination; ROC AUC = area under the receiver operating characteristic curve; IQR = interquartile range; LOF = Local Outlier Factor; DBSCAN = Density-Based Spatial Clustering of Applications with Noise; PNG = Portable Network Graphics. Concepts are explained in the relevant answers and glossary.

Table of contents

A - Purpose and overall architecture **4**

1. Purpose, users, inputs, outputs and limits . . . 4
2. Application layers and architecture . . . 5
3. Entry points and upload-to-download execution . . . 6
4. Workflow type and degree of autonomy . . . 8

B - Supervisor and execution control **9**

5. LangGraph nodes, edges, branches and loops . . . 9
6. What the supervisor actually decides . . . 10
7. Failures, incomplete outputs and invalid results . . . 10
8. Retries, fallbacks and recovery . . . 11
9. Dynamic tools, generated code and code repair . . . 12

C - Agent contract and shared state **13**

10. BaseAgent contract and execution behavior . . . 13
11. Nine-agent implementation inventory . . . 14
12. Every EMADSState field: producers and consumers . . . 19
13. Reducers, duplication, memory and checkpointing . . . 22
14. Files, sessions and concurrency risks . . . 23

D - Data processing and modeling **24**

15. Target and problem-type selection . . . 24
16. Preprocessing rules and assumptions . . . 25
17. Splits, data lifecycle and leakage . . . 26
18. Exact model families, principles and parameters . . . 27
19. Ranking, metrics, folds and shared splits . . . 28

20. Wilcoxon test: interpretation and limitations . . . 29

21. Optuna search, TPE, budgets and baseline comparison . . . 30

22. Final metrics, intervals and the downloadable estimator . . . 32

E - LLM decisions and explainability **33**

23. Complete LLM-call inventory and decision authority . . . 33

24. Parsing, provider failures, timeouts and outbound data . . . 34

25. SHAP: global and local explanations . . . 35

26. Permutation importance and Spearman agreement . . . 36

27. Meta-evaluation and all decision-confidence indicators . . . 37

F - Testing, limitations and interview defence **39**

28. Tests, mocks, executed evidence and missing coverage . . . 39

29. Discrepancies, ranked defects and a two-week improvement plan . . . 41

30. Interview presentation, short answers and follow-ups . . . 44

Glossary **47**

Verification evidence and environment **50**

Source symbol index **54**

A - Purpose and overall architecture

1. Purpose, users, inputs, outputs and limits

1. Simple explanation

EMADS (Explainable Multi-Agent Data Science System) automates a sequence of common data-science tasks: inspect a table, clean it, compare models, tune one model, measure its behavior, explain predictions, and produce a report. Its distinguishing design is a record of why important automated choices were made. Machine learning (ML) means learning patterns from examples rather than explicitly writing every prediction rule.

2. Actual implementation

Implemented: the Streamlit interface accepts a comma-separated values (CSV) file. The user can choose classification (predict a category), regression (predict a number), clustering (group similar rows), or anomaly detection (flag unusual rows). Auto-detection resolves only supervised classification/regression; unsupervised use requires an explicit choice. The intended audience suggested by the UI and README is someone exploring a tabular dataset, including a data-science practitioner or a less technical stakeholder reading the explanations. Actual user adoption was not measured.

Outputs include dashboard tabs, a cleaned CSV, comparison scores, a fitted estimator serialized as a pickle (.pkl), plots, decision records, and a Portable Document Format (PDF) report. Input is a local tabular file, not a database connection, image collection, streaming source, or natural-language task specification. There is no implemented prediction-serving endpoint or automatic production deployment.

3. Code evidence

• app/ui/streamlit_app.py:79, PROBLEM_TYPE_OPTIONS; :116, render_sidebar(); :622, render_report_tab(); :668, main().

• app/core/agents/data_understanding_agent.py:58, DataUnderstandingAgent.execute(); :299, _skip_target_for_unsupervised().

• app/core/agents/preprocessing_agent.py:184, cleaned-data output; app/core/agents/hyperparameter_agent.py:290, estimator persistence.

• README.md:5, overview; :163, explicitly scoped omissions. README intent is distinguished from verified runtime functionality.

4. Limitations or uncertainties

Partially implemented: generalization to arbitrary tabular CSVs. The code has size/cardinality heuristics and small-data failure paths; no universal compatibility guarantee is justified. It neither understands business intent nor verifies that a target represents the user's real question. It does not contain a calibrated trust probability. No hard end-to-end runtime bound is enforced. No CV (curriculum vitae) was supplied, so personal contribution and CV claims cannot be verified.

5. Short interview answer

EMADS is a local, explainable tabular-ML workflow. It automates analysis through reporting, while recording decisions and uncertainty indicators. It supports four problem types, but it is an exploratory prototype rather than a complete deployment or raw-data inference platform.

2. Application layers and architecture

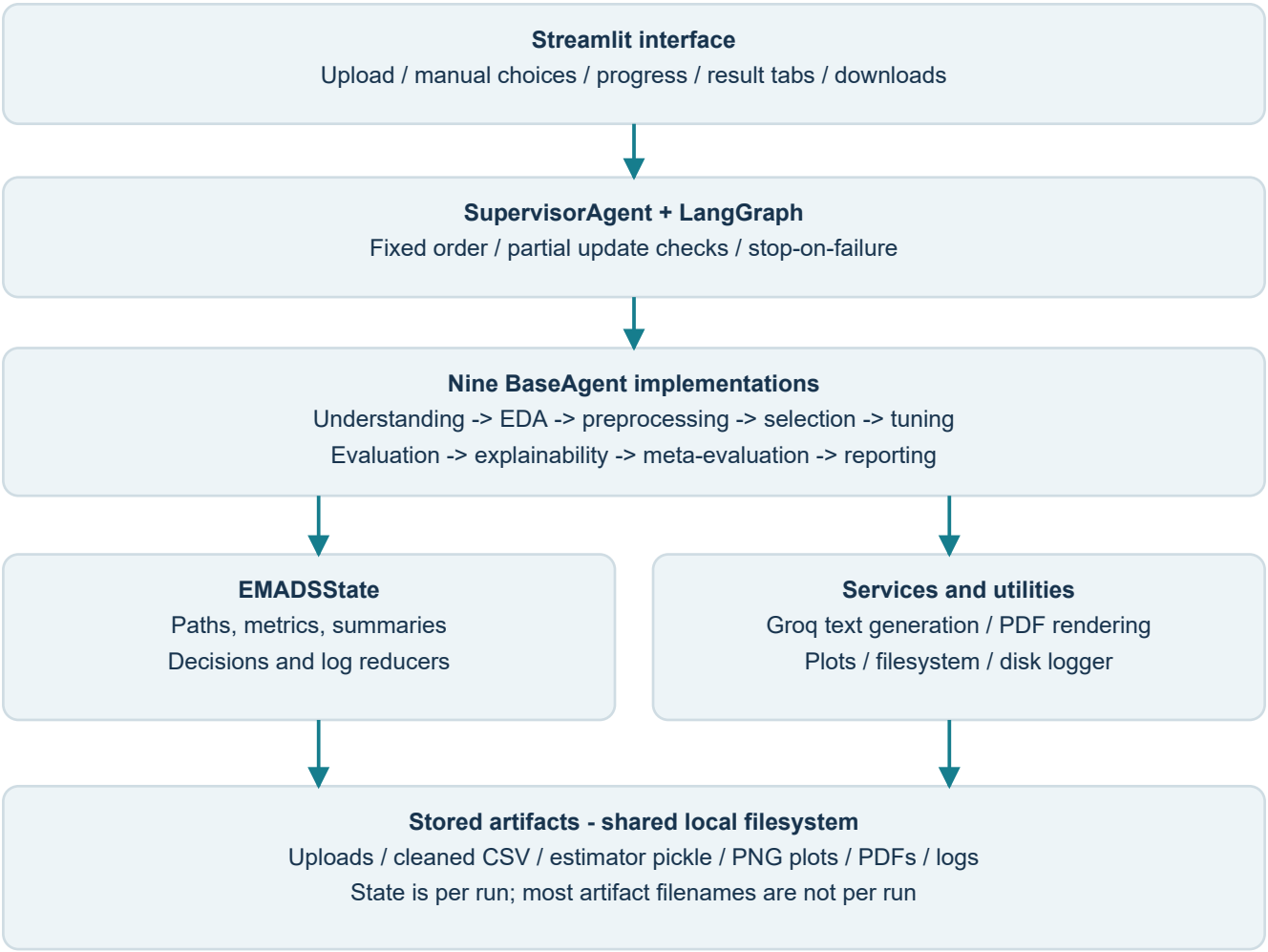
1. Simple explanation

A layer separates responsibilities: the interface handles interaction, orchestration controls execution order, agents implement analysis, state carries information, and services handle reusable external or rendering operations.

2. Actual implementation

The UI is Streamlit, with styling helpers in theme.py. SupervisorAgent compiles a LangGraph StateGraph: a directed graph whose nodes run functions and whose shared state is merged between nodes. The nine agents inherit BaseAgent. EMADSState is a typed dictionary, not an object that autonomously manages memory. LLMService wraps Groq text generation; PDFService formats supplied results with ReportLab. Plotting, file, and logger utilities support these components.

Stored artifacts are ordinary filesystem files. The workflow passes many file paths rather than maintaining one in-memory pandas DataFrame (a table object) throughout. There is no database-backed artifact registry or verified authorization boundary between users.



3. Code evidence

• app/ui/streamlit_app.py:209, run_pipeline_with_progress(); app/ui/theme.py:231, inject_theme() (see source index for exact current definition).

• app/core/supervisor/supervisor_agent.py:32, SupervisorAgent; :102, _build_workflow_graph().

• app/core/agents/base_agent.py:26, BaseAgent; app/core/state/emads_state.py:43, EMADSState.

• app/services/llm_service.py:36, LLMService; app/services/pdf_service.py:43, PDFService.

• app/utils/file_utils.py:16, ensure_dir(); app/utils/plot_utils.py:29, save_current_figure(); app/utils/logger.py:68, get_logger().

4. Limitations or uncertainties

Implemented separation of responsibilities does not imply independently deployed services, distributed execution, or isolated processes. These are Python modules in one application. Several module-level paths and caches are shared across runs. The diagram shows logical communication, not separate servers.

5. Short interview answer

The application has a presentation layer, fixed LangGraph orchestration, nine processing modules sharing a typed state, and services for Groq and PDF generation. Filesystem artifacts connect several stages; there is no database or distributed agent platform.

3. Entry points and upload-to-download execution

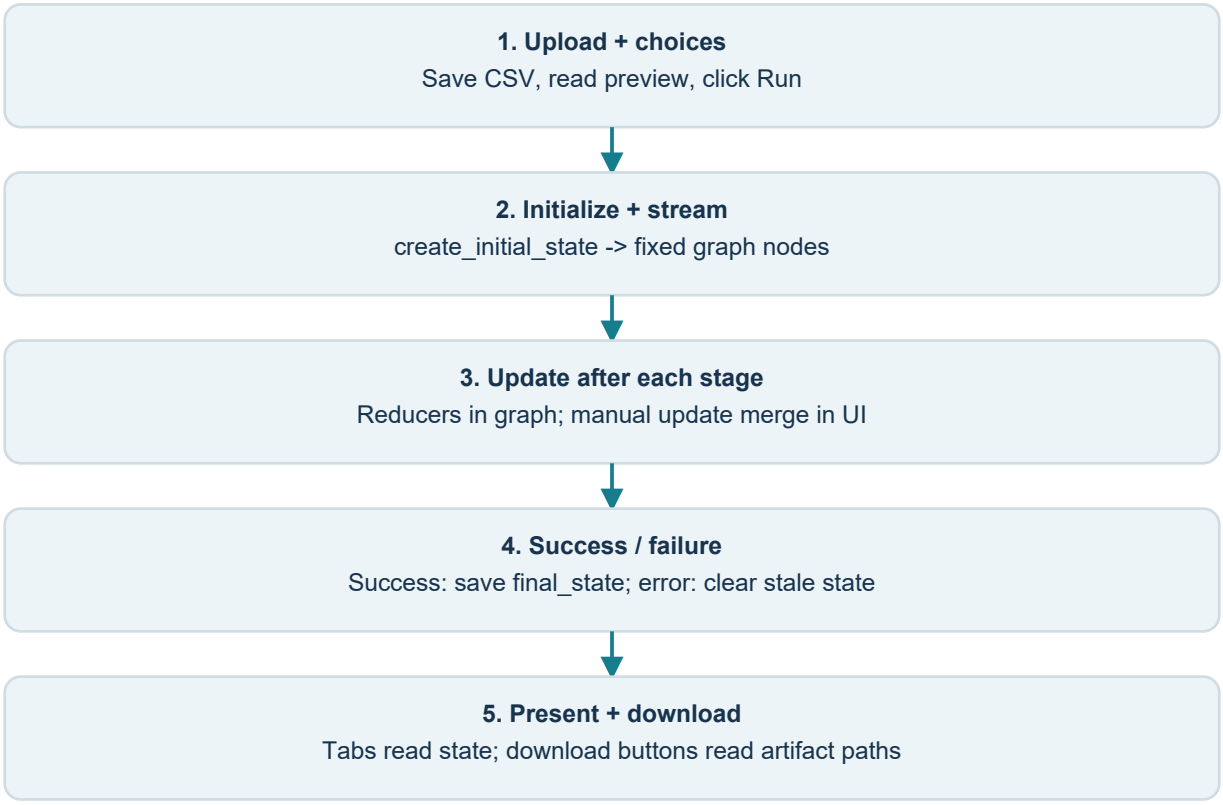
1. Simple explanation

An entry point is where execution starts. EMADS can start from a dashboard or from a Python script. The dashboard stores results for the current Streamlit session so later interface reruns can display them.

2. Actual implementation

Dashboard: streamlit run app/ui/streamlit_app.py executes module setup and main(). render_sidebar() writes the uploaded file to data/uploads/<provided name>, reads five rows for preview, and collects optional target/problem choices. Clicking Run creates a new initial state and SupervisorAgent. The UI uses supervisor.workflow.stream(initial_state), not SupervisorAgent.run_pipeline(). Each emitted node update is merged locally with _merge_stream_update(), and progress advances after completed stages. Successful results go into st.session_state['final_state']; failure removes previous results. Nine result tabs read the final dictionary. Downloads read report_path and model_path from disk when the page renders.

CLI (command-line interface): python run.py creates/overwrites data/mock_titanic.csv, constructs a small initial dictionary with target Purchased, and invokes run_pipeline(). Unlike the UI initializer, this script does not create a session_id, so reporting defaults to EMADS_Report_run.pdf. The root test_target_arbitration.py is a manual integration script using one bundled CSV without an explicit target; importing it executes the pipeline because it has no main guard.



3. Code evidence

• app/ui/streamlit_app.py:116, render_sidebar(); :209, run_pipeline_with_progress(); :244, graph stream; :296, _merge_stream_update(); :622, render_report_tab(); :668, main().

• app/core/state/emads_state.py:103, create_initial_state(); app/core/supervisor/supervisor_agent.py:115, run_pipeline().

• run.py:12, create_mock_dataset(); :33, main(); test_target_arbitration.py:1, module-level execution.

• app/core/agents/reporting_agent.py:72, generate_pdf_report() call and session-name fallback.

4. Limitations or uncertainties

Results are cleared after a failed run, but merely changing the uploaded file does not invalidate an earlier successful final_state until another run occurs. Progress is stage-level; no fine-grained trial progress is streamed. Reusing an artifact path can make an old session download a newer run's file. This is a risk inferred from shared paths, not a concurrency experiment performed here.

5. Short interview answer

Upload saves a local CSV, the UI initializes state and streams a fixed graph, then stores its final results in Streamlit session state. Tabs and downloads read those results and files. A separate CLI runs a bundled synthetic example through the same supervisor.

4. Workflow type and degree of autonomy

1. Simple explanation

A predefined workflow follows a program written in advance. A dynamically planned system decides which tasks or tools to execute at runtime. A workflow can contain limited adaptive choices without becoming a general planner.

2. Actual implementation

EMADS is a predefined workflow with localized adaptive decisions. pipeline_steps lists nine stages. Graph construction uses only unconditional add_edge calls. There is no add_conditional_edges, planner prompt, tool registry, action schema, or executor loop. Each class named Agent implements a bounded execute() method.

Rules choose preprocessing and problem-type behavior. Statistical calculations rank models and measure agreement. A large language model (LLM), which generates text from prompts, can suggest a target and resolve an inconclusive model comparison. Optuna chooses numeric configurations within coded search spaces. A user chooses a problem type/target or reruns after reading recommendations. Those are separate forms of control, not one autonomous mechanism.

3. Code evidence

• app/core/supervisor/supervisor_agent.py:40, pipeline_steps; :102, _build_workflow_graph().

• app/core/agents/base_agent.py:63, abstract execute() contract.

• app/core/agents/data_understanding_agent.py:166, _resolve_target_column(); app/core/agents/model_selection_agent.py:487, _compare_top_two().

• app/core/agents/hyperparameter_agent.py:250, bounded retry decision; app/core/agents/meta_evaluator_agent.py:66, audit-only execute().

4. Limitations or uncertainties

Implemented: automated execution and bounded adaptation. Absent: goal decomposition, runtime task planning, general tool selection, self-modifying code, and autonomous repair. Calling this a hybrid is defensible only if explicitly defined as a fixed workflow combining rules, statistics, optimization, and LLM assistance; it must not imply dynamic graph planning. README.md:17 correctly distinguishes the fixed graph from dynamic tool calling.

5. Short interview answer

EMADS is a fixed nine-stage workflow with conditional assistance inside some stages. Its autonomy is limited to predefined choices such as target suggestion, model tie-breaking, and parameter search; the supervisor does not plan a new workflow.

B - Supervisor and execution control

5. LangGraph nodes, edges, branches and loops

1. Simple explanation

A graph node performs a task; an edge specifies the next task. START and END mark entry and completion. A branch picks a route; a loop revisits work.

2. Actual implementation

SupervisorAgent creates StateGraph(EMADSState), registers each (step_name, agent) as a wrapped node, adds START to the first node, links adjacent entries with zip(), and adds the last node to END. compile() creates the executable workflow. Order is data_understanding, eda, preprocessing, model_selection, hyperparameter_optimization, evaluation, explainability, meta_evaluation, reporting.

There are zero conditional graph edges and zero graph loops. All supported types traverse all nine nodes. Internal branches do exist: supervised versus unsupervised paths; optional LLM arbitration; optional libraries; LinearRegression tuning skip; first-pass versus one widened Optuna retry; SHAP skip/failure handling; metric-specific rendering. These branches do not change graph topology. Candidate iteration, CV fitting, Optuna trials, Groq retry loops, and per-row explanation loops are ordinary Python/library loops inside nodes.

```
builder = StateGraph(EMADSState)
for step_name, agent in self.pipeline_steps:
    builder.add_node(step_name, self._wrap(step_name, agent))
builder.add_edge(START, self.pipeline_steps[0][0])
# Adjacent steps are connected in order; final step connects to END.
```

3. Code evidence

• app/core/supervisor/supervisor_agent.py:40, pipeline_steps; :102, _build_workflow_graph(); :108, START edge; :109, sequential edges; :111, END edge.

• app/core/agents/hyperparameter_agent.py:119, LinearRegression branch; :250, retry condition.

• app/core/agents/explainability_agent.py:83, unsupervised guard; app/services/llm_service.py:148, request retry loops.

4. Limitations or uncertainties

current_step is written into a returned update after node execution, so it describes a completed stage, not a reliable live in-progress marker. The UI separately anticipates the next step using a stepper. LangGraph capabilities such as conditional routing and checkpointing are not automatically used just because the package supports them.

5. Short interview answer

The graph is a linear chain built from one ordered list. START enters data understanding and reporting leads to END. Every branching or retry behavior is internal to a node; there are no graph-level loops or conditional routes.

6. What the supervisor actually decides

1. Simple explanation

A scheduler can enforce a schedule without reasoning about it. In EMADS, the supervisor's 'decision' is mostly a configuration fixed by the developer.

2. Actual implementation

The supervisor instantiates nine objects and fixes their order in pipeline_steps. _wrap() calls agent.execute_logged(state), converts the response to a dict, validates that four accumulator fields are lists, overwrites current_step with the step name, appends a supervisor success log, and returns the update. run_pipeline() invokes the compiled graph and wraps unexpected errors.

It cannot select a subset based on dataset content, skip a node dynamically, repeat a failed node, or run two nodes in parallel. One stage may deliberately skip a calculation inside its execute() implementation, such as SHAP for clustering, but that stage still runs. Scikit-learn n_jobs=-1 allows parallel model work; this is not parallel execution of agents.

3. Code evidence

• app/core/supervisor/supervisor_agent.py:37, __init__(); :63, ACCUMULATE_FIELDS; :65, _wrap(); :102, graph construction; :115, run_pipeline().

• app/core/agents/model_selection_agent.py:322, _score_candidates_supervised(), cross_val_score(..., n_jobs=-1).

• app/core/agents/explainability_agent.py:83, intentional unsupervised calculation skip.

4. Limitations or uncertainties

Implemented: ordering, wrapper-level validation, exception labeling, completion logs. Absent: a policy that learns routes, a supervisor LLM, a work queue, checkpoint resume, semantic output evaluation, or retries. Renaming the class would not alter its behavior; defend the actual code rather than inferring power from the name SupervisorAgent.

5. Short interview answer

The supervisor enforces a developer-defined order and wraps execution for logging and clearer errors. It does not choose tasks, tools, or routes at runtime. Parallelism, where present, is within model computation.

7. Failures, incomplete outputs and invalid results

1. Simple explanation

Fail-fast behavior stops dependent work after a failure. Structural validation checks shape or type; semantic validation checks whether a result is meaningful. EMADS implements only some of each.

2. Actual implementation

An exception in execute() is logged with a traceback by BaseAgent.execute_logged(), then re-raised. SupervisorAgent._wrap() converts it into RuntimeError("Pipeline stopped at step '<name>' (<type>): <message>") using 'from exc', preserving the original cause. UI streaming catches it, removes progress, shows an error, returns None, and main() clears final_state. The CLI propagates the failure.

Example of an actual executed test: an all-missing target causes a RuntimeError. Depending on inferred dtype, invalid classification can be rejected during data understanding; if it reaches preprocessing, dropping all unlabeled rows raises ValueError('No rows remain...'). The existing test asserts RuntimeError only, so it

does not prove the precise failing stage or wording.

The wrapper uses dict(agent.execute_logged(state) or {}). Thus None or an empty dictionary is accepted, a supervisor success message is added, and missing required outputs may fail later. A non-list logs value is rejected. A missing model_path or a NaN score is not generally validated by the supervisor. Supervised scoring catches individual candidate exceptions, records -infinity, and rejects a run if the first ranked score is -infinity. NaN (not-a-number) fold scores are not comprehensively checked; partial sklearn failures can therefore evade that sentinel check.

3. Code evidence

• app/core/agents/base_agent.py:76, execute_logged(); app/core/supervisor/supervisor_agent.py:65, _wrap(); :115, run_pipeline().

• app/ui/streamlit_app.py:282, streaming exception handler; :680, stale-state removal.

• app/core/agents/data_understanding_agent.py:391, _validate_target_for_modeling(); app/core/agents/preprocessing_agent.py:81, empty-after-target-drop check.

• app/core/agents/model_selection_agent.py:201, sorting/all-failed check; :322, candidate exception handling.

• tests/test_pipeline_edge_cases.py:28, test_empty_target_column_raises_clear_error() (executed and passed).

4. Limitations or uncertainties

Partial files written before a failure remain; there is no transaction or rollback. Unsupervised scoring converts exceptions to -1.0, indistinguishable from a degenerate silhouette result, and does not preserve per-candidate error details. Proposed: explicit output schemas, finite-score checks, post-dedup validation, and transactional artifact publication. None was implemented during this review.

5. Short interview answer

Unhandled stage errors stop the graph with a labeled RuntimeError and preserve the original cause. The UI clears stale results after failure. Validation is partial: it checks accumulator types and some agent inputs, but does not guarantee every successful update is complete or meaningful.

8. Retries, fallbacks and recovery

1. Simple explanation

A retry repeats an operation. A fallback substitutes a simpler behavior. Recovery after failure may also require restoring state; EMADS does not have a general restoration mechanism.

2. Actual implementation

Supervisor: no retry or resume; stop on first uncaught failure. LLMService: if no client, return supplied fallback text. With a client, try the selected model twice, then llama-3.1-8b-instant twice if distinct; exceptions sleep 0.5 seconds before continuing. Empty responses also trigger another attempt. There is no custom exponential backoff or explicit request timeout in this wrapper; SDK defaults may add their own behavior.

Tuning: first Optuna pass allows 20 trials and a nominal 60-second timeout. If the score gain over baseline is less than 0.01, at most one second pass widens selected bounds by 1.3, with remaining nominal budget from 90 seconds. The same study keeps the best trial across passes. LinearRegression bypasses tuning. A no-completed-trial fallback exists, but study.optimize() is called without catch=; an objective exception can propagate before the fallback is reached.

Other local fallbacks: a stratified train/test split may fall back to an unstratified split; optional boosted libraries can be absent; one model candidate can fail without stopping selection; SHAP/local explanations can fail without stopping reporting; CSV reading retries some encodings/separators only in data understanding.

3. Code evidence

• app/services/llm_service.py:127, _call_groq(); app/core/supervisor/supervisor_agent.py:65, _wrap().

• app/core/agents/hyperparameter_agent.py:202, first optimize(); :204, completed-trial check; :250, retry; :266, second optimize().

• app/core/agents/model_selection_agent.py:178, split fallback; :322, candidate fallback.

• app/core/agents/explainability_agent.py:102, SHAP catch; :154, local explanation catch; app/core/agents/data_understanding_agent.py:130, _load_dataset().

4. Limitations or uncertainties

Optuna timeout is checked between trials, not a preemptive kill of a running trial; retraining and other work also take time. Therefore 60/90 seconds are search budgets, not proven wall-clock limits. A retuned model can be worse than the original baseline and still replace it. The offline tests verify pipelines with deterministic fallback text, not real timeout, rate-limit, or retry behavior against Groq.

5. Short interview answer

Recovery is local and bounded: Groq text fallbacks, split/model/SHAP fallbacks, and one optional tuning retry. There is no supervisor restart, checkpoint resume, or automatic correction of the whole pipeline.

9. Dynamic tools, generated code and code repair

1. Simple explanation

Dynamic tool calling means choosing a callable operation from structured options at runtime. Generating code means producing executable source text; executing it requires an explicit executor. Text generation alone does not imply either capability.

2. Actual implementation

All analysis operations are hard-coded Python calls. Preprocessing directly calls pandas and sklearn. Model selection constructs a fixed dictionary of candidate estimators. The Groq request contains role/content messages, temperature, and max_tokens; it does not supply a tools schema, tool_choice, or function executor. LLM outputs are handled as strings, matched to column/model names, or rendered as prose.

Repository inspection found no implementation of generated Python execution, eval/exec of LLM output, code repair loops, or an action planner. Pickle loading exists for the application's own saved estimator; it is deserialization, not a generated-code repair feature, and pickle should only be loaded from trusted origins.

3. Code evidence

• app/services/llm_service.py:160, client.chat.completions.create() request.

• app/core/agents/preprocessing_agent.py:40, execute(); app/core/agents/model_selection_agent.py:287, _build_candidates().

• app/core/agents/data_understanding_agent.py:224, target output validation; app/core/agents/model_selection_agent.py:589, recommendation parser.

• app/core/agents/evaluation_agent.py:62, pickle.load(); README.md:17 and :163, dynamic-tool scope statements.

4. Limitations or uncertainties

Absent: dynamic orchestration/tool selection, generated-code sandbox, self-repair, and execution approval policy. Proposed only: if ever required, a constrained typed action registry would need validation and a separately designed execution boundary. It is not necessary to claim these features to defend the implemented workflow.

5. Short interview answer

EMADS agents use predefined library functions. The LLM selects between permitted names in a few decisions; it does not choose arbitrary tools or generate and execute Python code. I would describe this as modular workflow automation.

C - Agent contract and shared state

10. BaseAgent contract and execution behavior

1. Simple explanation

An abstract base class defines a common interface for different implementations. EMADS uses that interface so orchestration does not need to know the internal logic of each stage.

2. Actual implementation

BaseAgent inherits ABC and declares execute(state) abstract. Subclasses provide it. __init__(name) creates a named logger. decide(decision, reasoning, confidence) returns an AgentDecision dataclass with a timestamp; it does not validate correctness or enforce a confidence range. log(message) returns a formatted string for shared state. execute_logged() adds persistent start/end messages, logs a full traceback on exception, and re-raises it.

PartialEMADSState is simply an alias to dict, not a separately validated schema. Agents return only fields they change; they should return new accumulator entries, not all previous entries. Disk logging through self.logger and state['logs'] are distinct mechanisms. A return value must still travel through the supervisor wrapper before LangGraph merges it.

```
return {
    'metrics': metrics_payload,
    'agent_decisions': [decision],
    'logs': [self.log('Evaluation complete.')],
}
```

This is an illustrative shortened return shape; the complete evaluation return includes more fields.

3. Code evidence

• app/core/agents/base_agent.py:23, PartialEMADSState alias; :26, BaseAgent; :32, __init__(); :44, decide(); :58, log(); :63, execute(); :76, execute_logged().

• app/core/state/emads_state.py:35, AgentDecision; app/core/supervisor/supervisor_agent.py:79, wrapper conversion and validation.

• app/core/agents/evaluation_agent.py:151, actual partial update; app/utils/logger.py:68, get_logger().

4. Limitations or uncertainties

The base class enforces that an execute method exists, not a full input/output contract. It has no planner, tools, LLM client, retry, memory retrieval, or shared-state persistence. Confidence values and ownership conventions are not runtime-enforced. The wrapper accepts some incomplete results, as described in Q7.

5. Short interview answer

BaseAgent standardizes execution, decision records, and logging. Each subclass implements execute and returns a partial state update. Errors are logged and re-raised; the supervisor decides to stop, not the base class to repair them.

11. Nine-agent implementation inventory

1. Simple explanation

Each agent is a specialized processing stage. The following tables distinguish data it needs, calculations it performs, text generation it requests, and files it writes. This inventory describes executable code; a class name alone establishes none of these capabilities.

2. Actual implementation

All nine extend BaseAgent and return partial state updates. 'Decision/log' below means new agent_decisions/logs entries, not a replacement of their history. LLM means Groq via LLMService or deterministic offline fallback.

DataUnderstandingAgent

Aspect	Implementation
Purpose and inputs	Load dataset_path; read optional target_column and problem_type.
Outputs	schema_info, target_column, problem_type, decisions/logs, optional llm_arbitration_log entry.
Processing and libraries	pandas CSV/profile; naming, dtype, range and cardinality rules; quality warnings.
LLM	Only target fallback: column names in, single suggested name out.
Side effects	Disk/console logging; no cleaned-data or model write.
Validation	Required path; CSV exceptions; supervised classification >=2 classes and >=3 nonmissing rows/class; unsupervised >=30 raw rows.
Evidence	app/core/agents/data_understanding_agent.py:58 execute(); :130 load; :166 target; :330 type; :391 class validation; :410 quality.

EDAAgent

Aspect	Implementation
Purpose and inputs	Explore dataset_path using schema_info, target_column and problem_type.
Outputs	eda_stats, eda_summary, generated_plots, decision/log.
Processing and libraries	pandas describe(include='all'); Matplotlib/Seaborn plots; numeric-feature IQR outliers; at most six numeric distribution plots.
LLM	generate_eda_summary() narrates shape, column profiles, missingness and descriptive statistics; fallback summary exists.
Side effects	Plot PNG files under data/outputs/plots; logging.
Validation	Required path; plain pd.read_csv; plotting functions skip some inapplicable charts, but most exceptions propagate. Target excluded from feature outlier checks.
Evidence	app/core/agents/eda_agent.py:34 execute(); :137 fallback; app/utils/plot_utils.py:45 correlation; :121 detect_outliers_iqr().

PreprocessingAgent

Aspect	Implementation
Purpose and inputs	dataset_path, target_column unless unsupervised, problem_type, schema_info.
Outputs	preprocessing_report, preprocessed_data_path, selected_features, decisions/logs.
Processing and libraries	pandas dedup/drop/imputation/dummies; sklearn LabelEncoder and StandardScaler; numeric output CSV.
LLM	None. Strategies are explicit rules, not LLM choices.
Side effects	Overwrites data/outputs/clean_dataset.csv; logging. Fitted transforms are local variables and not persisted.
Validation	Required path/target; target column existence; nonempty data after dropping missing targets. No general post-transform feature/finite-value validation.
Evidence	app/core/agents/preprocessing_agent.py:40 execute(); :209 drop selection; :241 imputation; :255 categorical encoding.

ModelSelectionAgent

Aspect	Implementation
Purpose and inputs	preprocessed_data_path, problem_type, target_column for supervised use.
Outputs	candidate_models_results, selected_model_name, model_selection_summary, decisions/logs, optional arbitration entry. Does not save a fitted winner.
Processing and libraries	sklearn estimators/CV; optional XGBoost/LightGBM; SciPy Wilcoxon for top two supervised candidates; silhouette for unsupervised models.
LLM	Conditional supervised p>0.05 tie-break, parsed into one of the top two names; no unconditional model-selection narration call.
Side effects	Logging; candidate fits in memory, with library parallelism.
Validation	Required inputs; per-supervised-candidate exceptions recorded; all -infinity case rejected; incomplete NaN protection.
Evidence	app/core/agents/model_selection_agent.py:146 execute(); :287 candidates; :322 supervised scoring; :352 unsupervised scoring; :487 comparison; :589 parsing.

HyperparameterAgent

Aspect	Implementation
Purpose and inputs	preprocessed_data_path, problem_type, selected_model_name; supervised target; candidate_models_results baseline.
Outputs	best_hyperparameters, optimization_summary, model_path, decisions/logs.
Processing and libraries	Optuna TPE study maximizes CV accuracy/negative RMSE or full-data silhouette; sklearn and optional boosting estimators; one widened retry.
LLM	None.
Side effects	Fits estimator; overwrites data/outputs/models/selected_model.pkl; logging.
Validation	Required inputs; unknown model search-space/rebuild raises; no-completed-trial fallback after optimize; no global protection from objective exceptions.
Evidence	app/core/agents/hyperparameter_agent.py:105 execute(); :202 optimize; :250 retry; :440 search spaces; :535 rebuilding.

EvaluationAgent

Aspect	Implementation
Purpose and inputs	preprocessed_data_path, model_path, problem_type, target for supervised data.
Outputs	metrics, evaluation_plots, permutation_importance, decisions/logs.
Processing and libraries	pandas/pickle; sklearn metrics, clone, CV, permutation_importance; full-data fit_predict for unsupervised models.
LLM	None.
Side effects	Reads saved estimator; writes confusion/ROC or cluster scatter plots; logs metrics. Unsupervised in-memory refit is not re-saved here.
Validation	Required inputs; split fallback; ROC AUC exceptions return None; most evaluation errors propagate.
Evidence	app/core/agents/evaluation_agent.py:42 execute(); :163 unsupervised; :217 CV; :235 permutation; :267 AUC; :281 stability.

ExplainabilityAgent

Aspect	Implementation
Purpose and inputs	preprocessed_data_path, model_path, selected_model_name, problem_type, supervised target, permutation_importance.
Outputs	feature_importance, shap_plots, explainability_summary, local_explanations, explanation_agreement_score, decision/log, optional arbitration entry. SHAP numeric importance is computed locally but not retained as its own state field.
Processing and libraries	SHAP, NumPy/pandas, SciPy Spearman, sklearn split; native estimator importances; at most 100 global sample rows and three local examples.
LLM	General explanation narration; conditional disagreement narration if Spearman<0.3. Neither changes model or attribution calculation.
Side effects	Reads estimator; writes summary/waterfall PNGs; logs.
Validation	Required inputs; explicit unsupervised skip; catches SHAP and local-explanation errors; common-feature/NaN checks for correlation.
Evidence	app/core/agents/explainability_agent.py:51 execute(); :298 native; :311 SHAP; :350 agreement; :371 local explanations.

MetaEvaluatorAgent

Aspect	Implementation
Purpose and inputs	problem_type, metrics, explanation_agreement_score, llm_arbitration_log, agent_decisions.
Outputs	meta_evaluation dictionary, decision/log.
Processing and libraries	Python weighted formula: 0.4 evaluation + 0.3 explanation + 0.3 arbitration; threshold 0.6.
LLM	None.
Side effects	Logging only; does not rerun agents or write a model.
Validation	Defaults missing signals to neutral values; clamps components; no empirical calibration or full finite-value audit.
Evidence	app/core/agents/meta_evaluator_agent.py:66 execute(); :133 metric component; :157 explanation; :169 arbitration; :180 recommendation.

ReportingAgent

Aspect	Implementation
Purpose and inputs	Schema, target/type, summaries, plots, preprocessing, comparisons, selected model, parameters, metrics, explanations, decisions, arbitration, audit, session_id.
Outputs	report_path, final report-generation decision/log.
Processing and libraries	Builds structured payload; delegates to PDFService (ReportLab).
LLM	Executive conclusion from target, selected model and metrics. Prompt omits the meta-evaluation verdict.
Side effects	Writes reports/EMADS_Report_<session>.pdf; logging.
Validation	Many missing values default to empty text/list; PDF build exception becomes RuntimeError. No check that executive conclusion agrees with audit.
Evidence	app/core/agents/reporting_agent.py:26 execute(); :87 dataset summary; app/services/pdf_service.py:75 generate_pdf_report().

3. Code evidence

The evidence row in each table points to the corresponding implementation. The canonical order is app/core/supervisor/supervisor_agent.py:40, SupervisorAgent.pipeline_steps. Each agent's execute() return is the definitive list of state outputs; Q12 maps the complete state schema.

4. Limitations or uncertainties

Implemented does not mean every optional analysis succeeds: supervised SHAP errors are intentionally caught; a completed pipeline can lack SHAP plots. No agent has a general input/output schema validator. A no-target unsupervised explanation still creates an 'unknown' top-feature decision after skipping attribution, which is misleading wording rather than a useful explanation. All nine stages ran in the executed integration tests, but those tests do not assert every table entry or narrative claim.

5. Short interview answer

Each stage has explicit inputs, outputs and bounded operations. Some use an LLM, others are purely deterministic, and only target/model arbitration can alter those decisions. State and saved files connect stages; the base contract makes orchestration uniform.

12. Every EMADSState field: producers and consumers

1. Simple explanation

A shared state is a dictionary of the current run's results. A producer writes a field; a consumer reads it. Type annotations describe intended values but are not a comprehensive runtime validation system.

2. Actual implementation

EMADSState is TypedDict(total=False): every key is optional at the type level. Unless marked A below, a field uses replacement semantics. I means create_initial_state/UI/CLI input; DU data understanding; EDA exploration; PP preprocessing; MS model selection; HP tuning; EV evaluation; EX explanation; ME meta-evaluation; RP reporting; UI dashboard. These abbreviations are only for this table. 'Unused' means no meaningful downstream read found in inspected application source.

Field	Producer	Consumers and role
session_id	I	RP report filename; UI model download filename.
created_at	I	No downstream application read found.
current_step	I then supervisor	No substantive downstream read found; UI progress uses streamed event names.
dataset_path	I	DU, EDA, PP read CSV.
dataset_name	I	Stored metadata; no substantive downstream read found.
target_column	I then DU resolves	EDA, PP, MS, HP, EV, EX, RP, UI.
problem_type	I then DU resolves	EDA, PP, MS, HP, EV, EX, ME, RP, UI.
schema_info	DU	EDA profile context; PP drop flags; RP dataset summary; UI overview.
eda_summary	EDA	RP, UI; UI progress logging.
eda_stats	EDA	Retained but no later application consumer found; EDA uses local stats to build its prompt.

Field	Producer	Consumers and role
generated_plots	EDA	RP/PDF, UI EDA tab.
preprocessing_report	PP	RP/PDF, UI preprocessing tab.
preprocessed_data_path	PP	MS, HP, EV, EX, UI preview.
selected_features	PP	Stored but no downstream feature-selection consumer found.
candidate_models_results	MS	HP baseline; RP/PDF; UI comparison.
selected_model_name	MS	HP, EX, RP, UI. EV loads the estimator without needing this name.
model_path	HP	EV, EX, UI download. RP does not embed the estimator.
model_selection_summary	MS	RP payload and UI progress logging; verdict rendering mainly uses decisions/arbitration instead.
best_hyperparameters	HP	RP/PDF, UI model tab.
optimization_summary	HP	UI tuning summary; not forwarded by current ReportingAgent payload.
metrics	EV	ME, RP, UI.
evaluation_plots	EV	RP/PDF, UI.
permutation_importance	EV	EX agreement/cross-check; UI chart. Not directly forwarded as a numeric table to PDFService.
feature_importance	EX	UI chart/fallback. It is native importance, not the SHAP dictionary; not directly forwarded as a numeric table to RP.
shap_plots	EX	RP/PDF, UI.
explainability_summary	EX	RP/PDF, UI, UI progress logging.
local_explanations	EX	RP/PDF and UI per-row explanations.
explanation_agreement_score	EX	ME, RP payload, UI metric. Current PDF builders do not directly render the raw field as a dedicated metric.
report_path	RP	UI download/progress log; CLI console.
agent_decisions (A)	Every agent	ME inspects target decision; RP/PDF; UI verdict, overview and decision log.

Field	Producer	Consumers and role
logs (A)	Agents + supervisor	UI stream merge preserves them; root manual script/CLI may inspect state. No full state-log section is rendered in current PDF.
errors (A)	I empty list	No normal agent producer found. Exceptions stop execution instead of appending here.
llm_arbitration_log (A)	I; DU, MS, EX append	ME penalty; RP/PDF; UI arbitration/model verdict.
meta_evaluation	ME	RP/PDF; UI overview badge.

Ownership is a convention

target_column and problem_type intentionally start as user inputs and are later resolved by DU. current_step is written centrally. Most output fields have one main producing agent, but the comments' 'single owner' convention is not enforced by a permission system. Passing an unexpected extra key through LangGraph can differ from passing a plain dict directly to execute(); avoid claiming all runtime keys are accepted identically.

3. Code evidence

• app/core/state/emads_state.py:43, complete EMADSState declaration; :89, accumulators; :103, initializer.

• Producer returns: app/core/agents/data_understanding_agent.py:116; eda_agent.py:123; preprocessing_agent.py:194; model_selection_agent.py:271; hyperparameter_agent.py:315; evaluation_agent.py:151; explainability_agent.py:225; meta_evaluator_agent.py:111; reporting_agent.py:77. Paths in this sentence share prefix app/core/agents/.

• app/core/agents/reporting_agent.py:49, report_payload; app/ui/streamlit_app.py:296, stream update merge; :333 onward, tab consumers.

4. Limitations or uncertainties

Unused or partially consumed fields are not evidence of features: selected_features is a list of surviving columns, not an implemented feature-selection algorithm. errors is declared but not an implemented error-recovery queue. The state does not include fitted imputers/scalers, encoder mappings, train/test indices, dataset hashes, model package versions, or an external-call success flag. Proposed: persist those as explicit run provenance.

5. Short interview answer

EMADSState is an optional-key typed dictionary of run inputs and outputs. Most fields are replaced by their producing stage; four list fields accumulate entries. The schema also contains some metadata that is stored but not actively consumed, so I would not infer functionality from every declared field.

13. Reducers, duplication, memory and checkpointing

1. Simple explanation

A reducer specifies how an update combines with an existing value. Concatenating lists preserves earlier entries, but it does not remove duplicates. In-memory state disappears unless separately saved.

2. Actual implementation

The four fields agent_decisions, logs, errors and llm_arbitration_log use Annotated[List[...], operator.add]. For example, existing [A] plus update [B] becomes [A,B]. Agents should return [B], not [A,B]; returning the history again produces [A,A,B]. The supervisor adds a separate completion log for each successful stage in addition to its agent's entries. Those are intentional distinct messages, not proof of reducer corruption.

UI .stream() emits updates. _merge_stream_update() manually concatenates the same four fields while replacing others. This is necessary because the UI reconstructs a final dictionary from update events; it does not mean the same event is normally applied twice. Replaying events or switching stream modes without adapting the merger could duplicate entries. No identifiers or deduplication are applied.

Graph compilation supplies no checkpointer. There is no durable LangGraph checkpoint, database memory, history retrieval, resume token or cross-session reasoning memory. UI final_state persists only in a Streamlit session. Logs and files persist on disk, but these are artifacts, not a mechanism for restoring the full shared state.

3. Code evidence

- app/core/state/emads_state.py:89, Annotated reducers; :103, empty initial accumulator lists.
- app/core/supervisor/supervisor_agent.py:79, partial dict; :89, success log; :113, compile() without checkpointer.
- app/ui/streamlit_app.py:244, stream(); :296, _merge_stream_update(); :678, final_state session storage.
- app/utils/logger.py:62, file path; :68, persistent logging handlers.

4. Limitations or uncertainties

Implemented: append-style state and persistent log files. Absent: deduplicated events, state snapshots, checkpoint recovery, persistent agent memory. The current graph runs once and does not replay nodes, so accumulation alone does not imply duplication in normal operation. Proposed: event IDs and explicit retry semantics if replay is introduced. No duplication stress test was executed here.

5. Short interview answer

Reducers concatenate new list entries, so an agent must return only its new decisions and logs. Memory is per run/session; files survive, but there is no checkpoint-based resume or persistent reasoning memory.

14. Files, sessions and concurrency risks

1. Simple explanation

A session separates a user's interaction history. File isolation is separate: two sessions can still write the same filename. Safe multi-user operation requires both.

2. Actual implementation

create_initial_state() creates session_id from local datetime to second precision. Logger has a separate process-global timestamp and cache; UI reset_session() closes all cached handlers and creates a new logging session. Uploads use data/uploads/<uploaded_file.name>. Cleaned data, selected estimator, and many plot names are fixed paths shared by every run. Report paths use the timestamped session_id, except CLI's 'run' fallback. The model's download filename includes a model/session label, but the source file read remains selected_model.pkl.

The cleanup button keeps the five newest matching logs, reports and uploads using prune_old_files(); it is global and does not check session ownership. Existing data/models, data/plots and data/processed folders appear to be legacy locations; current processing writes data/outputs. .gitignore has legacy output patterns that do not cover all current output paths.

3. Code evidence

• app/core/state/emads_state.py:103, timestamp initialization; app/utils/logger.py:28, process globals; :34, reset_session().

• app/ui/streamlit_app.py:124, upload path/write; :190, cleanup button; :638, model download name.

• app/core/agents/preprocessing_agent.py:184, fixed CSV; app/core/agents/hyperparameter_agent.py:295, fixed pickle.

• app/utils/plot_utils.py:21, shared plot directory; app/core/agents/reporting_agent.py:73, PDF filename; app/utils/file_utils.py:30, pruning.

• .gitignore:45 onward, runtime-output patterns (legacy directories).

4. Limitations or uncertainties

High-impact inspection finding: concurrent runs can overwrite each other's files; an old session can read a new run's estimator or plots, and a report can embed mixed-run plots. Second-resolution IDs can collide. The code contains no application-level authentication, ownership checks, per-run directories, file locks, atomic artifact publish, or path-basename normalization at the write call. Framework filename handling and hosting protections were not tested, so actual path traversal or unauthorized remote access is not asserted. Cross-session leakage is a code-level risk, not an executed attack. Proposed: unique run directories, random identifiers, ownership authorization and isolated cleanup.

5. Short interview answer

Streamlit session state separates result dictionaries, but files are not isolated per session. The current shared filenames make concurrent use unsafe without further engineering. I would prioritize unique per-run artifact directories before multi-user deployment.

D - Data processing and modeling

15. Target and problem-type selection

1. Simple explanation

The target is what a supervised model predicts. Problem type determines whether that target is categorical or numeric, or whether no target is used. A column's shape is evidence, not knowledge of business intent.

2. Actual implementation

Target priority: (1) accept a supplied target if it exists; (2) take the first column whose stripped lowercase name is target, label, class, y, outcome or output; (3) use the final column as fallback and always request an LLM suggestion from the column names. Exact suggested-column membership after strip() is required; invalid text keeps the fallback. A supplied nonexistent target is not explicitly rejected here; it falls through to detection.

Common-name confidence is clamp(0.9-0.15*(matches-1),0.5,0.9). Blind fallback uses u=nonmissing distinct values/row count and clamp(0.55-0.4*u,0.15,0.55). A valid LLM/fallback response uses the suggested column's ratio and clamp(0.75-0.4*u,0.5,0.75). 'clamp' means restrict a number to a minimum and maximum. These are hand-designed scores, not fitted probabilities.

Explicit clustering/anomaly_detection sets target=None and requires at least 30 raw rows. Explicit classification/regression is accepted without inferring type. Otherwise: a nonnumeric target means classification; any floating dtype, numeric range >50, or >10 all-unique values means regression; remaining numeric targets with <=20 unique values mean classification; otherwise regression. Classification requires at least two nonmissing classes and at least three rows per class before preprocessing.

3. Code evidence

• app/core/agents/data_understanding_agent.py:27, naming constants; :166, _resolve_target_column(); :224, _arbitrate_target_column(); :299, _skip_target_for_unsupervised(); :320, row minimum; :330, _infer_problem_type(); :391, class checks.

• app/ui/streamlit_app.py:89, _no_obvious_target_suggested(), informational preview hint only; :116, manual controls.

4. Limitations or uncertainties

Float labels 0.0/1.0 are automatically treated as regression unless overridden. The range rule can misclassify discrete labels such as 0/100; low-cardinality counts can be classified when regression is intended. A five-row UI hint is not structurally equivalent to full-data evidence even though it reuses a formula. Deduplication can invalidate earlier class counts. Offline fallback returns a real last-column name, so the code labels it LLM arbitration and awards the higher confidence band without a live LLM response. This was reproduced with a 40-row synthetic table: recorded confidence 0.73 and an arbitration entry despite forced offline fallback.

5. Short interview answer

User input has priority, then known target names, then a last-column fallback with a constrained LLM suggestion. Problem-type inference is rule-based. The rules are useful defaults, but users must confirm intent, and the offline arbitration provenance needs correction.

16. Preprocessing rules and assumptions

1. Simple explanation

Preprocessing makes a table suitable for numerical estimators. Imputation fills missing cells, encoding converts categories to numbers, and scaling makes numeric magnitudes comparable. Each transformation embeds assumptions.

2. Actual implementation

Duplicates: remove exact duplicate rows and reset indices. Missing targets: discard unlabeled rows for supervised work. Column dropping: parse quality-warning messages for constant/high-cardinality column names and drop those columns; also drop columns with >60% missing cells, except the target. High-cardinality warnings refer to nonnumeric columns with >50 categories; unique numeric identifiers are not generally detected.

Missing features: numeric columns get the median, falling back to zero when the median is missing; other columns get a literal 'Missing' category. Categoricals with <=10 distinct values use pd.get_dummies(drop_first=True). Higher-cardinality categoricals use LabelEncoder on strings, unless already dropped through the >50-category warning. Nonnumeric targets are label-encoded. StandardScaler fits numeric feature columns to zero mean and unit population variance. Boolean dummy columns are generally not included in select_dtypes(['number']) scaling.

Outliers: EDA detects feature outliers with the 1.5*IQR rule, where IQR (interquartile range) is the 75th minus 25th percentile. Preprocessing does not remove, cap, or robustly transform those outliers. The cleaned CSV is saved; encoders, category mappings, imputers and scaler are not saved.

3. Code evidence

• app/core/agents/preprocessing_agent.py:40, execute(); :209, _select_columns_to_drop(); :241, _impute_missing_values(); :255, _encode_categoricals(); :169, scaler fit_transform.

• app/core/agents/data_understanding_agent.py:410, _detect_quality_issues(); app/utils/plot_utils.py:121, detect_outliers_iqr(); app/core/agents/eda_agent.py:64, numeric feature filtering.

4. Limitations or uncertainties

Dropping all >50-category strings conflates identifiers and useful predictors. Label encoding imposes an arbitrary order on nominal categories. Duplicate rows can be legitimate independent events; deleting them can change class prevalence. Missingness can itself be predictive. Date/free-text features are not semantically engineered. No general infinity handling, zero-feature guard, post-dedup class validation, or unseen-category inference path exists. Missingness/encoding/scaling use the full data before splitting (Q17). Proposed improvements should be evaluated on data appropriate to the business problem, not claimed as automatically superior.

5. Short interview answer

Preprocessing uses deterministic median/placeholder imputation, cardinality-based encoding, deduplication and standardization. Outliers are flagged, not treated. The main engineering gap is persisting and fitting transforms inside the training/CV pipeline.

17. Splits, data lifecycle and leakage

1. Simple explanation

A training set fits a model, a validation set compares choices, and a test set estimates performance after choices are fixed. Cross-validation (CV) repeats training/validation across folds. Leakage occurs when information from supposed evaluation rows influences training or selection.

2. Actual implementation

Raw CSV -> full-data profiling/EDA -> full-data deduplication, dropping, imputation, encoding and scaling -> cleaned CSV. Model selection then makes an 80/20 split with random_state=42, stratifying classification when feasible. Only the 80% training portion enters selection CV. Tuning reconstructs the same split and searches with up to three folds. Final supervised fitting uses that 80%. Evaluation reconstructs the same holdout, predicts its 20%, and performs fresh up-to-five-fold CV on the training portion. Local explanations recreate the same test split; global SHAP instead samples from all cleaned features.

Candidates share the same splitter object/data and deterministic seed, so their CV folds are paired within model selection. The tuning and evaluation CV splitters use different maximum fold counts, so those scores are not directly interchangeable with selection scores. There is no separate stored validation dataset or nested CV. Unsupervised selection, tuning and evaluation all fit/score the full cleaned dataset; there is no held-out assessment there.

3. Code evidence

• app/core/agents/preprocessing_agent.py:114, imputation; :131, encoding; :169, scaling; :184, cleaned CSV.

• app/core/agents/model_selection_agent.py:178, split; :395, CV splitter; app/core/agents/hyperparameter_agent.py:172, split; :410, tuning CV.

• app/core/agents/evaluation_agent.py:72, split; :217, fresh-clone CV; :163, full-data unsupervised evaluation.

• app/core/agents/explainability_agent.py:312, full-data sample; :371, local explanations/test split.

4. Limitations or uncertainties

Leakage is implemented, not hypothetical: medians, category vocabulary, drop decisions and scaler statistics use future holdout rows and CV validation folds. A synthetic example: if the future test subset contains a very large income value, the global scaler mean/variance already incorporates it before CV. Even unsupervised transforms must be fitted only on each training fold for a strictly independent supervised evaluation. Final estimator fitting excludes test rows, but the full processing chain does not. The audit's use of test metrics is post-hoc; repeated user decisions based on that test can further consume its independence. Proposed: split raw data first and use a fitted sklearn Pipeline/ColumnTransformer within CV.

5. Short interview answer

The estimators respect a reproducible 80/20 supervised split, but preprocessing happens earlier on all rows, so the end-to-end evaluation is not fully leakage-free. Fixing this requires fold-local preprocessing and a saved inference pipeline, not just changing the model split.

18. Exact model families, principles and parameters

1. Simple explanation

Different models make different assumptions. Linear models learn weighted combinations; trees learn decision rules; ensembles combine trees; clustering and anomaly methods learn structure without target labels.

2. Actual implementation

The initial candidate configurations are predefined; XGBoost/LightGBM appear only if imports succeed. These are implemented classes, not hypothetical candidates.

Family / problem	Class and principle	Initial settings in selection
Classification linear	sklearn.linear_model.LogisticRegression; weighted features mapped to class probabilities.	max_iter=1000, random_state=42.
Regression linear	sklearn.linear_model.LinearRegression; least-squares linear prediction.	Library defaults; tuning skipped.
Classification/regression tree	sklearn.tree.DecisionTreeClassifier / DecisionTreeRegressor; recursive feature thresholds.	random_state=42; no initial depth cap.
Classification/regression forest	sklearn.ensemble.RandomForestClassifier / RandomForestRegressor; average/vote across randomized bootstrapped trees.	n_estimators=100, random_state=42, n_jobs=-1.
Classification/regression boosted trees	xgboost.XGBClassifier through internal XGBClassifierWithLabelEncoding / XGBRegressor; sequential gradient boosting.	random_state=42, verbosity=0; classification eval_metric='logloss'. Remaining settings are installed-library defaults.
Classification/regression boosted trees	lightgbm.LGBMClassifier / LGBMRegressor; gradient-boosted decision trees.	random_state=42, verbosity=-1.
Clustering	sklearn.cluster.KMeans; minimize within-cluster squared distances to centroids.	n_clusters=3, random_state=42, n_init=10.
Clustering	sklearn.cluster.DBSCAN; density-connected clusters with noise points.	eps=0.5, min_samples=5.
Anomaly	sklearn.ensemble.IsolationForest; unusual rows isolate with short random-tree paths.	random_state=42, contamination='auto'.
Anomaly	sklearn.neighbors.LocalOutlierFactor; compare local density to neighbors.	novelty=False, contamination='auto'; default neighbors from installed sklearn.

Important tunable parameters: C controls inverse regularization strength; max_depth limits tree complexity; min_samples_split/leaf constrain splits; n_estimators controls tree count; learning_rate controls boosting step size; subsample controls sampled training fraction; KMeans n_clusters chooses group count; DBSCAN eps is a neighborhood radius; contamination sets anomaly fraction/threshold; LOF n_neighbors controls local scale. Q21 lists exact searched ranges.

XGBoost's classifier wrapper encodes labels to contiguous 0..k-1 during fit, reverses predictions, and exposes original classes_. This supports labels such as 1..5. It is duplicated in selection and tuning modules.

3. Code evidence

• app/core/agents/model_selection_agent.py:80, optional imports/wrapper; :287, _build_candidates().

• app/core/agents/hyperparameter_agent.py:53, duplicate XGBoost wrapper; :440, parameter search; :535, final model construction.

• tests/test_xgboost_label_encoding.py:50, non-contiguous classes; :73, contiguous classes; :107, tuned pickle behavior (all three executed).

4. Limitations or uncertainties

Library-default parameters vary by installed version; dependencies have lower bounds rather than a reproducible lockfile. No neural network, support vector machine, forecasting model, text model or automatic feature generator is registered. DBSCAN and LOF(novelty=False) do not offer the same reusable out-of-sample predict API as supervised estimators. The first-fit XGBoost fix is real, but a review probe found that refitting the same wrapper with labels 1,2,3 fails because stale _label_encoder affects classes_ during super().fit(). No source fix was applied.

5. Short interview answer

Supervised selection compares linear, tree, forest and optional XGBoost/LightGBM models. Unsupervised selection compares KMeans/DBSCAN or IsolationForest/LOF. Families and initial settings are coded explicitly; only their performance comparison and later tuning are automated.

19. Ranking, metrics, folds and shared splits

1. Simple explanation

A metric turns model behavior into a number. A fair comparison evaluates candidates on the same data splits. A random seed makes pseudorandom choices reproducible under comparable software and execution conditions.

2. Actual implementation

Supervised models: cross_val_score returns per-fold accuracy for classification or negative root mean squared error (RMSE) for regression. Negative RMSE is used because sklearn scorers follow 'larger is better'. Results store mean_score, std_score (NumPy population standard deviation), scoring_metric and fold_scores. They are sorted descending. The highest mean is the default winner, subject to the top-two LLM arbitration.

Selection classification CV uses StratifiedKFold(shuffle=True, random_state=42) with min(10, minimum training class count), at least two folds, when minimum count >=2. Otherwise it uses shuffled KFold with min(10, training row count), at least two. Regression uses that KFold rule. The preceding 80/20 split uses seed 42 and stratification if feasible; a ValueError causes an unstratified fallback. All candidate estimators use the same training rows and deterministic splitter, enabling paired fold comparisons. Candidate training itself can remain hardware/library sensitive.

Unsupervised candidates fit_predict the entire cleaned table and get silhouette_score(X,labels). A single label gets -1; exceptions also get -1. Standard deviation is reported as zero because only one fit is scored. Anomaly labels -1 and +1 are treated as two groups; DBSCAN noise -1 is included as a label for silhouette. No ground-truth anomaly accuracy is computed.

3. Code evidence

• app/core/agents/model_selection_agent.py:146, execute(); :178, split; :199, scoring; :201, sorting; :322, _score_candidates_supervised(); :352, _score_candidates_unsupervised(); :395, _build_cv_splitter().

• app/core/agents/model_selection_agent.py:51, CV_FOLDS=10; :52, RANDOM_STATE=42; :487, top-two test.

4. Limitations or uncertainties

Accuracy can hide failure on rare fraud cases; weighted metrics later do not correct the selection objective. Plain random splits are inappropriate for some time/group-dependent problems. Small classes reduce folds below ten, so '10-fold' wording is not always accurate. Selection summary text hardcodes CV_FOLDS even when fewer folds are used. Zero unsupervised std is not evidence of stability. Candidate exceptions and true poor silhouettes can collapse to the same value; NaN CV scores need explicit rejection. Proposed: problem-aware objectives, group/time split options, real fold-count provenance and repeated robustness assessment.

5. Short interview answer

Candidates share reproducible training and CV splits and are ranked by mean accuracy or negative RMSE. Unsupervised candidates use full-data silhouette. The ranking is a practical comparison under those metrics, not a guarantee that the winner is best for every business objective.

20. Wilcoxon test: interpretation and limitations

1. Simple explanation

The Wilcoxon signed-rank test compares paired numeric observations. It ranks the magnitudes of nonzero pair differences and asks whether positive and negative differences are consistent with a symmetric distribution centered at zero. A p-value measures incompatibility with that null hypothesis under the test assumptions; it is not the probability that one model is right.

2. Actual implementation

_compare_top_two() takes fold_scores from the two highest mean-scoring supervised candidates. It calls scipy.stats.wilcoxon(best_folds, second_folds) with default two-sided alternative, zero handling and method. Missing/fewer-than-two fold arrays skip the test. A ValueError maps to p=1.0. If p>0.05, Groq is asked to arbitrate using interpretability and cost; otherwise the raw winner remains. Exactly p=0.05 takes the non-arbitrated path. Selection confidence is min(0.99,1-p), which is a display transformation, not a calibrated probability of superiority.

With n nonzero untied paired differences and an exact two-sided test, the smallest attainable p is 2/2^n. For five pairs that is 0.0625, so significance at 0.05 cannot be reached even with all differences in one direction. Ten pairs permit 0.001953125. Review probes reproduced both values with method='exact'. But actual reduced-fold runs, zeros/tied absolute differences, and SciPy's default method selection affect applicability. Equal differences can produce warnings; in the installed SciPy, all-zero default inputs returned p=1.0, and the comparison correctly triggered arbitration in the probe.

Assumptions include meaningful pairing, independent pairs for the usual reference distribution, and an appropriate symmetric distribution of differences. CV folds share overlapping training data, so their scores/differences are dependent. Additionally, testing only the top two after screening several candidates introduces selection/multiple-comparison concerns. Thus the test is a heuristic decision aid here, not rigorous

proof that a model is globally superior.

3. Code evidence

• app/core/agents/model_selection_agent.py:1, mathematical-fold-floor discussion; :51, ten-fold cap; :226, confidence transformation; :487, _compare_top_two(); :519, wilcoxon call; :529, p-value threshold.

• tests/test_model_selection_arbitration.py:48, override test; :69, invalid response; :146, execute-level recommendation test (executed).

• tmp/pdfs/inspect_review.py, exact-test and all-zero probes; tmp/pdfs/inspection_results.json, recorded values. These are review evidence, not production modules.

4. Limitations or uncertainties

Non-significance means insufficient evidence of a difference under this test, not equivalence or interchangeability. Formal equivalence needs a meaningful predefined tolerance and a suitable procedure. Ten folds do not eliminate dependence or guarantee significance, and small datasets often use fewer. Catching every ValueError as an exact tie could mislabel malformed inputs. No finite p-value validation exists. Proposed: report effect sizes and uncertainty, state dependence limitations, and treat secondary selection as a documented policy rather than statistical certainty.

5. Short interview answer

I use paired fold scores with Wilcoxon to detect an inconclusive top-two comparison, then optionally use secondary criteria. A large p-value does not prove equivalence, and 1-p is not the probability the winner is correct. Shared CV training data limits the formal interpretation.

21. Optuna search, TPE, budgets and baseline comparison

1. Simple explanation

Hyperparameters are configuration values chosen before fitting, such as tree depth. Optuna evaluates candidate configurations. Its Tree-structured Parzen Estimator (TPE) sampler uses previous trial outcomes to favor promising parameter regions, rather than exhaustively enumerating every combination.

2. Actual implementation

Only selected_model_name is tuned. optuna.create_study(direction='maximize', sampler=TPESampler(seed=42)) creates an in-memory study. Each supervised objective constructs a fresh estimator and returns the mean accuracy or negative RMSE from up-to-three-fold CV on the training 80%. Unsupervised objectives fit_predict all cleaned features and return silhouette, or -1 for a one-label fit. LinearRegression is fitted directly on the training split and skips tuning.

Initial search spaces

Model	Searched parameters
LogisticRegression	C: 0.001 to 10, logarithmic.
DecisionTree	max_depth: 2..20; min_samples_split: 2..20; min_samples_leaf: 1..10.
RandomForest	n_estimators: 100..400 step 50; max_depth: 3..20; min_samples_split: 2..15.
XGBoost	n_estimators: 100..400 step 50; max_depth: 3..10; learning_rate: 0.01..0.3 logarithmic; subsample: 0.6..1.0.
LightGBM	n_estimators: 100..400 step 50; max_depth: 3..12; learning_rate: 0.01..0.3 logarithmic.
KMeans	n_clusters: 2..10.
DBSCAN	eps: 0.1..2.0; min_samples: 3..15.
IsolationForest	n_estimators: 50..300 step 50; contamination: 0.01..0.3.
LocalOutlierFactor	n_neighbors: 5..50; contamination: 0.01..0.3.

First optimize(): at most 20 trials, nominal timeout 60s. Baseline is the selected candidate's mean_score from model selection. If best_score-baseline<0.01 and >1s remains from a nominal 90s combined search budget, run at most 20 more trials in the same study. Retry multiplies upper bounds by 1.3 for C, depth/count/sample bounds, clusters, neighbors and eps; learning_rate, contamination and subsample retain their original ranges. Integer bounds are rounded to a valid step by _widen_int_high(). Best parameters across both passes are fitted and pickled.

3. Code evidence

• app/core/agents/hyperparameter_agent.py:34, limits; :105, execute(); :198, study creation; :202, first pass; :250, retry condition; :266, retry pass; :290, fit/persistence.

• app/core/agents/hyperparameter_agent.py:403, _get_baseline_score(); :410, CV splitter; :425, integer widening; :440, search spaces; :535, rebuilding.

4. Limitations or uncertainties

The 0.01 threshold is an absolute scorer difference, not a universal 1% relative improvement; its units differ between accuracy, RMSE and silhouette. Baseline uses up-to-ten-fold CV; tuning uses up-to-three folds, so their difference mixes configuration improvement with split differences. Best trial may underperform the baseline yet still be saved, and some default configurations are outside the search space (e.g., unrestricted default trees). No baseline retention guarantee exists. Timeout is not preemptive, and final fitting lies outside the search budget. The no-complete-trial fallback cannot catch an objective exception that aborts optimize() first. No pruning, durable study storage, nested validation or adaptive model-family search is configured. Proposed: identical baseline folds, baseline as an explicit trial, finite scores, and a defined failure policy.

5. Short interview answer

Optuna tunes only the previously selected family with TPE and coded parameter ranges. It allows one widened retry when the initial gain is small. The search is bounded in trials and nominal time, but the comparison to baseline and wall-clock guarantees need tighter engineering.

22. Final metrics, intervals and the downloadable estimator

1. Simple explanation

Metrics summarize predictions; confidence intervals try to express uncertainty about an estimate. A saved estimator is not necessarily a complete raw-data prediction system: it may expect already transformed features.

2. Actual implementation

Classification holdout metrics: accuracy = correct/n; precision = TP/(TP+FP); recall = TP/(TP+FN); F1 is their harmonic mean. precision_recall_fscore_support(..., average='weighted', zero_division=0) averages class metrics using true-class support. A confusion matrix counts actual/predicted combinations. ROC AUC (area under the receiver operating characteristic curve) uses predict_proba: second-class probabilities for binary data, weighted one-versus-rest for multiclass, returning None on exceptions.

Regression: mean absolute error (MAE), mean squared error (MSE), RMSE=sqrt(MSE), and R-squared (R2), the improvement over predicting the test mean in squared-error terms. R2 can be negative and is unreliable/undefined on very small or degenerate test targets.

Evaluation clones the fitted estimator and computes fresh up-to-five-fold training CV mean and population std. The displayed interval is mean +/- 1.96*std/sqrt(5), regardless of the actual number of folds used. It is an approximate normal-theory interval on CV scores, not a prediction interval for an individual new observation and not an interval on holdout accuracy. For regression it is in negative-RMSE units.

Unsupervised evaluation refits with fit_predict on the full table. It calculates silhouette (larger is better), Davies-Bouldin (smaller suggests tighter/separated groups), Calinski-Harabasz (larger suggests separation), and cluster sizes or anomaly counts/rate. Single-label fits get silhouette -1 and other indices None. DBSCAN noise counts as a distinct label, including in n_clusters_found.

The saved .pkl contains the fitted final estimator on supervised training rows or the unsupervised full dataset. It does not contain preprocessing, raw schema handling, feature encoders/scaler, target-label map for preprocessed string targets, split indices or a model card. The XGBoost wrapper alone does retain its internal numeric-label encoder. Raw strings/missing columns cannot automatically be processed by the download.

3. Code evidence

• app/core/agents/evaluation_agent.py:42, execute(); :93, interval formula; :98, regression metrics; :110, classification metrics; :163, unsupervised; :217, CV clone; :267, _compute_roc_auc().

• app/core/agents/hyperparameter_agent.py:129, linear-model fit; :290, final fit and pickle; app/core/agents/preprocessing_agent.py:157, target encoder; :169, scaler.

• app/ui/streamlit_app.py:622, report/model download and loading instruction.

4. Limitations or uncertainties

Partial independence only: holdout rows are excluded from estimator fitting and candidate scoring, but influence full-data preprocessing (Q17). Training CV after selection/tuning is not an independent external validation estimate. Fold dependence, post-selection bias, fixed sqrt(5), and normal approximations weaken the interval interpretation. No calibration, per-class report, precision-recall curve, external dataset evaluation, inference API or unseen-category path is implemented. Proposed: complete serialized pipeline and corrected uncertainty reporting. Do not load untrusted pickle files.

5. Short interview answer

The final supervised metrics use a reproducible holdout and a fresh CV stability check. They are useful diagnostics, but preprocessing leakage and the approximate interval formula limit their statistical interpretation. The download is an estimator expecting processed columns, not an end-to-end raw-data pipeline.

E - LLM decisions and explainability

23. Complete LLM-call inventory and decision authority

1. Simple explanation

An LLM can write an explanation without controlling execution. To determine its authority, trace where its returned text is consumed: does it change a selected name, or only appear in a report?

2. Actual implementation

Provider: Groq SDK. Default model string: llama-3.1-8b-instant. Allowed configured strings: llama-3.1-8b-instant, llama-3.3-70b-versatile, mixtral-8x7b-32768, gemma2-9b-it. An allowed environment model overrides even an explicitly supplied allowed constructor model. The allowlist is a local code list, not evidence those remote models are currently available. Requests use system/user messages, max_tokens=600, temperature=0.2 unless model tie-breaking sets 0. No tools schema or structured JavaScript Object Notation (JSON) response mode is configured.

Call and trigger	Prompt/input and requested output	Decision effect
Target fallback; no user target/name match	Senior-data-scientist prompt asks ONLY one column name. Input: full list of column names. Fallback: final column. Temperature 0.2.	Exact valid name becomes target; changes downstream training.
EDA; every EDA execution	Senior-data-scientist prompt asks 4-6 plain-English bullets on shape, missingness, types, imbalance and preprocessing. Input: file path, shape, target, profile and descriptive statistics.	Narration only.
Model tie-break; supervised Wilcoxon p>0.05	Senior-ML-engineer prompt asks 2-4 sentences selecting between two named candidates using interpretability/cost. Input: model names, mean metric, p-value. Temperature 0.	Parsed recommendation can promote runner-up; actual selected_model_name changes.
Explanation disagreement; valid Spearman<0.3	Prompt asks why SHAP/permutation disagree, what it means for trust, and says not to pick a side. Input: top five importance values from each method and correlation.	Narration/log only; does not change model or feature ranking.
Main explanation; every EX execution	Nontechnical explanation prompt requests 4-6 bullets. Input: selected model, top five native features/values, SHAP top names, top five permutation values.	Narration only, including after unsupervised attribution is skipped.

Call and trigger	Prompt/input and requested output	Decision effect
Executive conclusion; every reporting execution	Director-style prompt requests 2-3 sentences about deployment readiness. Input: target, selected model and metrics.	Report text only. Meta-evaluation is not included in this prompt.

There are six call sites. A typical run requests EDA, main explanation and conclusion; target/model/disagreement calls are conditional. LLMService creates separate service instances for agents but uses the same configuration logic. No LLM is called by preprocessing, tuning, evaluation or meta-evaluation.

3. Code evidence

• app/services/llm_service.py:45, model constants; :53, configuration; :127, _call_groq(); :202, generate_eda_summary(); :246, generic generate_summary().

• app/core/agents/data_understanding_agent.py:235, target prompt; eda_agent.py:107, EDA call; model_selection_agent.py:534, tie-break prompt; explainability_agent.py:127, disagreement prompt and :170, main summary; reporting_agent.py:30, conclusion. Paths share app/core/agents/ prefix.

4. Limitations or uncertainties

No live API calls were made in this review; remote model availability, latency, cost, response quality and real provider failures remain unverified. Prompts use role descriptions, not evidence of expert reliability. Reporting asks about readiness from metrics without receiving the audit or known preprocessing weaknesses, so it can overstate trust. The README statement that the LLM 'only steps in' at uncertainty points is incomplete because narration calls are routine. Proposed: explicit provenance/status, consistent audit-aware prompts and conservative evidence-grounded language.

5. Short interview answer

There are six LLM call sites. Two can change a permitted decision: target suggestion and top-two model arbitration. EDA, explanation and conclusion calls write prose; the actual training and metric calculations remain Python/library operations.

24. Parsing, provider failures, timeouts and outbound data

1. Simple explanation

Validation checks that generated text fits a permitted result. It does not establish truth. A fallback keeps execution working when a service is unavailable, but must be distinguishable from an actual service response.

2. Actual implementation

Target parsing: strip whitespace and require exact column membership. Quoted names or explanations fail that check and keep the final-column fallback. Model parsing: lowercase, find the first substring 'recommend', and choose whichever candidate name appears earliest afterwards; without that word search the whole response. Neither candidate found means keep the raw top scorer and record parse failure. A valid-looking name can still be semantically the wrong interpretation.

Reproduced example: 'I do not recommend RandomForest; choose LogisticRegression.' returns RandomForest. The parser does not understand negation. Model fallback text deliberately names/recommends the top scorer, so offline fallback appears as a parsed recommendation. Target fallback similarly returns a valid column and is logged as LLM arbitration in the higher confidence band (Q15). No structured result distinguishes offline, API-failed, empty, and live responses.

_call_groq catches Exception, logs it, sleeps 0.5s and retries; after attempts it returns fallback text, or a '[Groq API Error]' string when none exists. No app-level request deadline is configured. Installed groq 1.5.0 declares a 60s timeout (5s connect) and two SDK retries by default; SDK retries can multiply the wrapper attempts. These are inspected defaults, not exercised provider timing. Rate limits are not treated specially in application code.

What can leave locally: column names; dataset filename/path; dtypes and missing counts; descriptive statistics including categorical 'top' values; target/model names; metric dictionaries; feature names/importance values; and p-values/correlation. The code does not upload the whole CSV or plots as attachments, but aggregate text can still contain sensitive categories or identifiers. This review's tests suppress dotenv loading, force offline clients and block socket connections; no live data was transmitted.

3. Code evidence

• app/core/agents/data_understanding_agent.py:235, prompt; :245, strip/membership; app/core/agents/model_selection_agent.py:589, parser.

• app/services/llm_service.py:127, fallback/retries; :160, request; :202, EDA input assembly.

• app/core/agents/eda_agent.py:77, describe(include='all'); app/core/agents/explainability_agent.py:485, explanation prompt; app/core/agents/reporting_agent.py:30, conclusion prompt.

• venv/Lib/site-packages/groq/_constants.py:9, timeout and :10, retries (installed dependency evidence, not project source).

• tmp/pdfs/inspect_review.py and inspection_results.json, parser/offline probes.

4. Limitations or uncertainties

Credential values were not read or included. The API-key-source logger can name an environment variable that differs from the actual selected key because source resolution is separate from validation. There is no prompt-injection filtering for malicious column names, schema-based JSON parsing, consent/minimization control in the UI, or response audit flag. Proposed: typed enum outputs, exact schema validation, explicit call-status provenance, bounded timeouts and sensitive-category redaction. A user-visible explanation is not a security boundary.

5. Short interview answer

Outputs are restricted to known names, but the free-text model parser is imperfect and can misread negation. Service failures return deterministic text. Aggregate schema/statistical information can leave through Groq, so I would add explicit provenance, structured validation and data-minimization controls before broader use.

25. SHAP: global and local explanations

1. Simple explanation

SHAP (SHapley Additive exPlanations) attributes a model's output to input features relative to a baseline, using ideas from cooperative-game Shapley values. It explains model behavior under explainer assumptions; it does not identify causes in the real world.

2. Actual implementation

Global: load the saved estimator and cleaned table; sample min(100,n) feature rows from the entire dataset using seed 42. For DecisionTree, RandomForest, XGBoost and LightGBM names, instantiate shap.TreeExplainer(model) without an explicit background dataset. Tree defaults determine its background/feature-dependence assumptions and output units. Other names use shap.Explainer(model, sample), supplying the same sampled rows as background/masking reference and evaluation data; the library selects a compatible explainer. Call explainer(sample).

For ordinary outputs, SHAP values have rows x features. Multiclass can have rows x features x classes. The code collapses 3D values with mean(abs(values), axis=2), constructs a new Explanation with zero base_values for the bar-plot API, then averages absolute contributions across rows for each feature. Those zero baselines are plotting placeholders after aggregation, not genuine per-class prediction baselines. It saves a summary bar plot and returns a top-five feature list and a numeric dictionary locally. Native feature_importance stored in state remains a separate measure.

Local: reconstruct the held-out split, obtain predictions, choose one misprediction if available, then sample remaining positions up to three total with seed 42. For regression, 'misclassified' actually means absolute residual greater than one test-target standard deviation. Tree explainers use model-only construction; non-tree local explainers use the selected local sample as background. For per-class output, select the predicted class using model.classes_. Render waterfall plots and save original cleaned-table row index, actual, predicted, error flag and path. Explanations refer to encoded/scaled features, not original raw units.

3. Code evidence

• app/core/agents/explainability_agent.py:28, SHAP_SAMPLE_SIZE; :38, TREE_MODEL_NAMES; :83, unsupervised skip; :298, native importance; :311, _compute_shap(); :350, agreement; :371, _compute_local_explanations().

• app/core/agents/explainability_agent.py:328, multiclass collapse/placeholder baseline; :440, class-specific local output selection.

• tests/test_explainability_unsupervised.py:61 onward, explicit skip tests (executed); supervised SHAP was exercised by integration runs but not asserted in detail.

4. Limitations or uncertainties

Global SHAP is not restricted to the holdout; different global/local non-tree backgrounds can change explanations. No feature-dependence policy or output-unit normalization is explicitly configured. Native multiclass coef_ handling takes only abs(coef_[0]), not an average over all classes. The decision text may say top feature is 'derived from SHAP' while the chosen top_feature actually comes from native importance. SHAP exceptions are caught, so passing pipeline tests does not prove SHAP success for every estimator. Unsupervised SHAP is skipped by design here; that is an implementation choice, not a universal impossibility of explaining anomaly scores.

5. Short interview answer

EMADS uses tree-specific or automatically selected SHAP explainers for supervised models, summarizes mean absolute contributions, and shows a few held-out waterfall examples. The explanations are descriptive, depend on background assumptions, and currently use inconsistent global versus held-out data scope.

26. Permutation importance and Spearman agreement

1. Simple explanation

Permutation importance asks how much predictive performance drops when one feature is shuffled. Spearman correlation compares relative ordering by ranking values. Agreement between methods is a useful diagnostic, not a proof that an explanation is true.

2. Actual implementation

EvaluationAgent calls sklearn.inspection.permutation_importance on X_test/y_test with n_repeats=10, random_state=42 and n_jobs=-1. Classification scoring is accuracy; regression scoring is R2 (not the negative RMSE used for model selection). It stores each feature's mean score drop sorted descending; negative drops are retained. Per-repeat uncertainty is not retained in state.

ExplainabilityAgent builds common_features from names appearing in both the global SHAP importance dictionary and permutation dictionary. With >=2 names it calls scipy.stats.spearmanr on the corresponding

importance values, returns None on errors/NaN, otherwise preserves the [-1,1] correlation. Spearman handles tied values through ranks; a constant vector can make it undefined. If agreement<0.3, an LLM writes possible reasons and a trust caveat. It is instructed not to pick a side and does not recompute either method or change the model.

Feature-name alignment is compatible because both use columns from the same cleaned CSV. Data scope is not identical: global SHAP uses a sample from the full table; permutation uses held-out rows. Units are different (attribution magnitude versus performance drop), so comparing ranks is more meaningful than directly subtracting magnitudes. But the two methods also answer different questions and use different dependence assumptions.

3. Code evidence

• app/core/agents/evaluation_agent.py:235, _compute_permutation_importance(); :252, decision from the highest score drop.

• app/core/agents/explainability_agent.py:121, agreement trigger; :127, caveat prompt; :311, SHAP sample; :350, _agreement_score(); :238, native/permutation top-feature note.

• README.md:103, claim that both methods run on the same test set (contradicted by these code paths).

4. Limitations or uncertainties

Correlated features can substitute for each other, reducing individual permutation drops while receiving SHAP attribution. Noise, small holdout samples, weak models, class imbalance, background choices and interactions can also cause disagreement. The LLM may list such hypotheses, but cannot establish the actual cause from top-five lists and one coefficient alone. Correlation with only two features is especially coarse; high agreement can occur for equally misleading rankings. Proposed: matched held-out samples/background policy, repeated uncertainty, grouped permutation for related features, and a minimum evidence threshold before drawing conclusions.

5. Short interview answer

Permutation importance measures held-out score drops; SHAP measures attribution magnitudes. I compare their rankings with Spearman on shared feature names. Disagreement prompts a caveat and investigation, not an automatic claim that one explanation is wrong.

27. Meta-evaluation and all decision-confidence indicators

1. Simple explanation

A heuristic indicator is a hand-designed summary. A calibrated probability would need evidence that outcomes assigned, for example, 80% confidence are correct about 80% of the time under a defined event and population. EMADS has indicators, not that calibration study.

2. Actual implementation

Meta score = 0.4*E + 0.3*X + 0.3*A. E is classification accuracy clamped [0,1], regression R2 clamped [0,1], or (silhouette+1)/2 clamped [0,1]; missing E defaults 0.5. X is (Spearman+1)/2 clamped [0,1], default 0.5 if unavailable. A=max(0.4,1-0.1*arbitration_count). At score>=0.6, the verdict says report as reliable; below it, recommend manual review. Recommendation priority is low target confidence (<0.5), then explanation disagreement (<0.3), then generic review. The audit does not loop back or change an estimator. Its own AgentDecision confidence is 1.0 because the formula execution is deterministic; that is separate from meta_evaluation.confidence_score.

Decision-level formulas

Here clamp(v,a,b) means min(b,max(a,v)); proportions/ranges are defined by the local data. These formulas are implementation rules, not validated probability models.

Decision	Formula or implemented rule
Explicit target/type or unsupervised target skip	1.0 configuration fact.
Common target name	clamp(0.9-0.15*(matching names-1),0.5,0.9).
Last-column fallback / valid suggested target	clamp(0.55-0.4*u,0.15,0.55) / clamp(0.75-0.4*u,0.5,0.75), u=distinct/rows.
Inferred nonnumeric classification	clamp(0.97-0.3*u,0.7,0.97).
Inferred numeric classification	clamp(0.95-0.35*(unique/20),0.6,0.95).
Inferred regression	clamp(0.6+0.35*u,0.6,0.97).
EDA outlier decision	With outliers: clamp(0.5+0.4*affected_feature_share,0.5,0.9). Otherwise clamp(0.6+0.3*min(rows,1000)/1000,0.6,0.9).
Preprocessing duplicate removal / unsupervised target-step skip	1.0, describes completed operation/configuration.
Dropped columns	Average each dropped column's missing ratio if >0.6, otherwise 1.0; clamp(0.6+0.35*average,0.6,0.98).
Imputation	clamp(0.95-0.5*average_imputed_column_missing_ratio,0.5,0.95).
Encoding	clamp(0.6+0.3*one_hot_share,0.6,0.9).
Scaling	clamp(0.6+0.3*min(std_ratio,10)/10,0.6,0.9).
Supervised model comparison	min(0.99,1-p) when Wilcoxon available. Otherwise 0.9 for one candidate, or min(0.95,0.5+10*absolute_score_gap).
Unsupervised model comparison	0.9 for one candidate; otherwise clamp(0.5+2.25*silhouette_gap,0.5,0.95).
Tuning improvement	0.6 without baseline; clamp(0.5+10*max(gain,0),0.4,0.95) with baseline. No-completed-trial fallback uses constant 0.5.
Tuning skipped / retry unnecessary or budget-exhausted	1.0 for stated execution/configuration fact; an executed retry uses the improvement formula.
Supervised evaluation consistency	clamp(0.9-3*absolute(test_score-CV_mean),0.4,0.9), using accuracy or negative RMSE.
Unsupervised evaluation	clamp((silhouette+1)/2,0.3,0.9).
Permutation top-feature decision	clamp(5*top_drop,0.3,0.9) if positive, else 0.3.

Decision	Formula or implemented rule
Explanation agreement	max(0,Spearman) when computable. Otherwise clamp(0.5+0.4*relative_native_top_gap,0.5,0.9), then +0.05 for matching permutation top (cap .95) or -.1 for disagreement (floor .4).
Report generation / meta formula application	1.0 for successful execution fact, not confidence in scientific validity.

3. Code evidence

• app/core/agents/meta_evaluator_agent.py:29, weights; :66, execute(); :133, E; :157, X; :169, A; :180, recommendations.

• app/core/agents/data_understanding_agent.py:166, target; :330, type; eda_agent.py:82, outliers; preprocessing_agent.py:122, imputation; :138, encoding; :166, scaling; :227, drop confidence.

• app/core/agents/model_selection_agent.py:226, 1-p; :434, fallback margin; :454, unsupervised margin; hyperparameter_agent.py:338, improvement; :349, retry decisions.

• app/core/agents/evaluation_agent.py:140, stability; :200, unsupervised; :252, permutation; explainability_agent.py:196, agreement confidence; :458, fallback. Agent paths share app/core/agents/ prefix.

4. Limitations or uncertainties

Statistics-derived does not mean statistically calibrated. 1-p is not P(model is best), Spearman is not P(explanation is correct), and the meta weights/threshold are manual. A single bad target can be masked by good downstream metrics; the target confidence is not directly a weighted component. Offline attempted arbitrations still incur penalties and may claim endorsement. The meta component maps Spearman to (r+1)/2 while EX confidence floors r at zero: the same r=0.5 becomes .75 versus .5. A weak model can pass the audit threshold because missing explanations get neutral credit. EX top-feature text can cite SHAP while using native importance. Reporting conclusion omits audit context; its deterministic fallback also claims valid cross-validation alignment without checking it. These are trust/wording defects, not proof of malicious behavior. Proposed: rename confidence to explicit evidence indicators, show components/provenance, and evaluate calibration only after defining correctness events.

5. Short interview answer

The system records interpretable confidence indicators and a weighted audit, but they are not calibrated probabilities. I can explain every formula and its inputs; I would not claim an 80% score means an 80% chance the entire analysis is correct.

F - Testing, limitations and interview defence

28. Tests, mocks, executed evidence and missing coverage

1. Simple explanation

A real integration test exercises connected components. A mock substitutes controlled behavior for part of the system. Passing tests establishes only the properties actually asserted, under the tested environment.

2. Actual implementation

The existing tests/ suite contains 23 unittest tests across six files. All were executed: 23 passed, zero failures, zero errors, zero skips, 56.163 seconds for the suite. This is a review execution time under the harness, not a product benchmark. Python 3.13.0 and installed versions are listed in the evidence appendix.

Exact command from the repository root:

```
& '.\venv\Scripts\python.exe' tmp/pdfs/run_existing_tests.py `
  *> tmp/pdfs/existing_tests.log
```

The review harness uses unittest.defaultTestLoader.discover(root/tests, top_level_dir=root), then TextTestRunner(verbosity=2). It changes cwd to tmp/pdfs/test_run to isolate relative artifacts, suppresses dotenv loading, patches LLMService._resolve_api_key to None, blocks socket connections, uses joblib threading rather than subprocess workers, and limits native numerical-library thread pools to one. Project source is unchanged. These environment controls avoid live APIs and overwriting existing output files; they mean the default process-parallel execution mode was not tested.

Existing file / count	Real components exercised	Mocked or restricted / what assertions establish
tests/test_explainability_unsupervised.py / 5	Fits real KMeans, DBSCAN, IsolationForest and LOF; real EX execute/read pickle.	_compute_shap mocked and asserted not called; main LLM text mocked. Confirms empty attribution outputs and deliberate skip log.
tests/test_model_selection_arbitration.py / 6	Real parser, comparison/Wilcoxon and execute control flow; temporary CSVs.	Groq output mocked; execute tests mock candidate dictionary and CV score arrays. Confirms winner switching, parse fallback and truthful reasoning text, not real model quality.
tests/test_pipeline_edge_cases.py / 3	Full supervisor pipeline for small regression/classification data; all-missing target failure.	Offline LLM via review harness. Asserts metric keys/RuntimeError, not exact failure stage, unbiased performance or every explanation.
tests/test_target_arbitration_confidence.py / 3	Real structural confidence formulas and target validation against synthetic tables.	LLM suggestion mocked. Confirms differing ratios change confidence and scores stay in the specified band; does not validate calibration.
tests/test_unsupervised_pipeline.py / 3	Full clustering/anomaly graph on two synthetic blobs; too-small dataset failure.	Offline LLM. Confirms metric/model-family presence and minimum-row rejection, not anomaly correctness or report visual quality.
tests/test_xgboost_label_encoding.py / 3	Real candidate CV and Optuna/XGBoost training; pickle reload and predictions.	Only global offline LLM restriction. Optional-library skip guard present, but no tests skipped here. Confirms first fit/tuning supports labels 1..5 and ordinary 0..2 classes.

Additional review probes, separate from the 23 tests

Exact command: & '.\venv\Scripts\python.exe' tmp/pdfs/inspect_review.py *> tmp/pdfs/inspection.log. Final rerun exited 0. Probes reproduced the negation-parser error, offline target confidence/provenance issue, and second-fit XGBoost wrapper error. They also checked exact Wilcoxon p minima and all-zero default behavior. See tmp/pdfs/inspection_results.json. The initial probe process completed its JSON but PowerShell reported exit 1 while wrapping stderr warnings; rerunning with Python stderr merged into stdout removed that shell ambiguity. It is not counted as an existing-suite failure.

3. Code evidence

• Test definitions: tests/test_explainability_unsupervised.py:61; test_model_selection_arbitration.py:48 and :146; test_pipeline_edge_cases.py:28; test_target_arbitration_confidence.py:35; test_unsupervised_pipeline.py:43; test_xgboost_label_encoding.py:50 and :94.

• tmp/pdfs/run_existing_tests.py, harness controls; tmp/pdfs/test_results.json and existing_tests.log, exact run evidence; tmp/pdfs/inspect_review.py and inspection_results.json, diagnostic evidence.

• test_target_arbitration.py:1, root manual script not included in tests/ discovery and not executed here because it uses a bundled dataset and adds no required coverage beyond the isolated tests.

4. Limitations or uncertainties

Untested or insufficiently asserted: live provider behavior, rate-limit/timeouts, malicious/negated/ambiguous recommendations beyond one probe, concurrency, user isolation, cleanup ownership, finite scores, empty-feature matrices, separator/encoding consistency, post-dedup class minima, objective-failure fallback, hard runtime, inference on new raw data, confidence calibration, CV dependence/CI coverage, native-vs-SHAP narrative alignment, and supervised SHAP correctness across every model. No coverage percentage, broad benchmark or personal bug-fix authorship is claimed. Existing integration tests generated PDFs, but this guide's QA does not validate every historical/test report layout.

5. Short interview answer

I have 23 passing existing tests in an isolated offline run. They establish key pipeline paths, arbitration behavior and first-fit label encoding, but not production correctness. I separate mock-based control-flow evidence from real model execution and explicitly list missing coverage.

29. Discrepancies, ranked defects and a two-week improvement plan

1. Simple explanation

A documentation claim should match executable behavior. A useful review ranks problems by their effect on validity, privacy and usability, then proposes changes with concrete acceptance criteria. Proposed work is not current capability.

2. Actual implementation

Nine historical PDFs in reports/ were opened and text-extracted with PyMuPDF; all contain the generic 'grounded in real statistics' claim, statistical verdict and confidence-audit headings. Six contain a Local Explanations heading. Historical outputs are not tied to the inspected commit by an artifact manifest; their detailed run metrics are not used as current-code evidence. No separate thesis report or CV was supplied. No private dataset rows or historical metric values are reproduced.

Claim/source	Verified discrepancy or qualification
README.md:90; app/services/pdf_service.py:156; historical report cover text	'Confidence ... never hardcoded' / grounded in statistics overstates reality: several constants and arbitrary clamps/weights exist, and no calibration is established (Q27).
README.md:103	Says SHAP and permutation use the same test set and EX computes both. EV computes permutation on holdout; EX computes global SHAP on a full-data sample.
README.md:95	'LLM only steps in' at uncertain decisions omits unconditional narration calls and EX disagreement narration. Model-selection unconditional narration described in some docs is absent from current execute().
app/ui/streamlit_app.py:67 and :205	UI says eight specialized agents; pipeline_steps contains nine.

Claim/source	Verified discrepancy or qualification
app/ui/streamlit_app.py:433; app/services/pdf_service.py:278	The no-arbitration branch says a statistical test found a significant difference, even when no Wilcoxon comparison was applicable (for example unsupervised scoring). Absence of arbitration is not evidence of significance.
Model-selection / evaluation wording	Hardcoded '10-fold' and '5-fold' text can disagree with reduced splitter counts; CI still divides by sqrt(5). No test of actual interval coverage exists.
app/core/agents/hyperparameter_agent.py:36 and test-plan timeout language	'Hard safety cap' / never >90s is not a preemptive wall-clock bound. Trials/final fitting can exceed nominal search budgets.
Model download caption, app/ui/streamlit_app.py:647	Pickle loading works as deserialization, but the caption omits the required external preprocessing and lost target mapping.
README.md:163, 'no cloud deployment'	Local configuration supports Streamlit cloud secrets; this does not prove a deployment exists. Neither a live deployment nor its absence can be established solely from the repository.
Test plan versus tests	The test-plan document proposes broader per-agent, retry, timing and UI checks than the six existing test files implement. A plan is not executed evidence.

Ranked defects and limitations (review judgment)

1. High - evaluation validity: preprocessing leakage and optimistic post-selection diagnostics. Evidence: preprocessing_agent.py:114, model_selection_agent.py:178, evaluation_agent.py:217.

2. High for multi-user hosting - shared artifact filenames/global logging allow cross-run contamination; no app-level access control. Evidence: streamlit_app.py:124; preprocessing_agent.py:184; hyperparameter_agent.py:295; logger.py:34.

3. High for trust claims - non-calibrated confidence, offline arbitration misattribution, native-vs-SHAP wording and audit-blind conclusions. Evidence: data_understanding_agent.py:245; explainability_agent.py:194; meta_evaluator_agent.py:66; reporting_agent.py:30.

4. High for reuse - model export lacks preprocessing, target mapping and provenance. Evidence: hyperparameter_agent.py:290; preprocessing_agent.py:157.

5. Medium - free-text parser negation, NaN score handling, all-failed unsupervised ambiguity, no post-dedup or zero-feature guard. Evidence: model_selection_agent.py:322, :352, :589; preprocessing_agent.py:40.

6. Medium - tuning baseline/folds differ, worse tuned result retained, objective exceptions can bypass fallback, time limits not hard. Evidence: hyperparameter_agent.py:198, :250, :290, :403.

7. Medium - approximate CI uses wrong fold denominator on small data; unsupervised noise metrics and one-feature plotting corner cases. Evidence: evaluation_agent.py:95 and :163; plot_utils.py:104.

8. Medium - XGBoost wrapper cannot safely refit nonzero labels on the same instance; both duplicated definitions need the same correction. Evidence: model_selection_agent.py:100; hyperparameter_agent.py:66; review probe.

9. Lower - incorrect agent-count text, legacy ignore patterns, unused state fields, report payload omissions, loosely constrained dependency versions. Evidence: streamlit_app.py:67; .gitignore:45; emads_state.py:43; reporting_agent.py:49; requirements.txt:1. File basenames in this ranking refer to the full paths cited in their respective answers and the source index.

Proposed two-week plan - none of these changes were made

Days	Proposed work	Acceptance criterion
1-2	Split raw supervised rows first; implement fold-local Pipeline/ColumnTransformer and record indices.	Test-only distribution changes cannot affect fitted training transforms; no preprocessing fit touches validation/test rows.
3	Save complete inference artifact, label mapping, schema and versions.	Reload and predict raw held-out rows with missing/unseen categories; predictions match pre-save outputs.
4	Unique run directories, random IDs, safe filenames, atomic writes, isolated logger ownership.	Concurrent synthetic runs cannot read/overwrite each other's artifacts; old-session downloads remain correct.
5	Output schemas, finite scores, empty-data/features and post-dedup checks; fix XGBoost refit.	Meaningful failure messages and regression tests cover each reproduced fault.
6	Structured LLM decisions with explicit offline/error/live status; exact enum validation.	Negation/ambiguous prose cannot silently choose a model; offline fallback never claims live endorsement.
7	Same CV baseline for tuning, default trial retention and explicit trial-error policy; deadline reporting.	Chosen configuration cannot lose to baseline on the comparison folds; timeout wording matches observed semantics.
8	Correct interval/fold reporting; rename confidence indicators; show audit components and caveats.	Report uses actual fold counts and no probability/calibration claim without evidence.
9	Align SHAP/permutation data, multiclass importance and narrative source; audit-aware conclusions.	Feature-source labels match the ranked values and report conclusion includes review flags.
10	Re-run tests; add isolated UI/report smoke checks; document versions, claims and remaining scope.	Source-linked documentation, preserved test logs, visually checked reports and reproducible commands.

This is a realistic prioritized ten-working-day scope if access to requirements and representative permitted datasets is available; it is not a promise of statistically calibrated trust or production certification. Dynamic planning, code repair, cloud deployment and persistent agent memory are not proposed as necessary fixes.

3. Code evidence

• README.md:17, :90, :95, :103, :163, :229; app/services/pdf_service.py:115, report-cover builder; app/core/agents/reporting_agent.py:49, payload.

• EMADS V2 — Plan de test complet et datasets recommandés.md:18, retry/timing plan; :52, two-minute target. This is a planning document, not a benchmark.

• tmp/pdfs/inspection_results.json, nine historical report names/page counts and generic-text checks. The evidence appendix lists exact files without reproducing their private data.

4. Limitations or uncertainties

Severity depends on use: shared files are especially serious under concurrent hosting, while a single-user offline demo has a smaller exposure. No exploit or external deployment was tested. The personal contribution, CV wording, historical report generation commit and independent dataset performance remain unresolved. Existing dirty runtime files predated this review and were preserved.

5. Short interview answer

The strongest current contribution is transparent, modular workflow automation. The highest-priority improvements are leakage-free preprocessing, isolated artifacts, complete model export and honest decision provenance. I can defend implemented behavior while distinguishing it from planned production engineering.

30. Interview presentation, short answers and follow-ups

1. Simple explanation

A credible interview explanation states the problem, the implemented mechanism, the evidence, and its limits. Do not claim authorship or results that cannot be supported. The following first-person wording is a presentation template about the project, not a verification of your individual contribution.

2. Actual implementation

Approximately 90-second project presentation

EMADS is an explainable workflow for tabular machine learning. A user uploads a CSV and can specify the target and problem type. The application then runs nine predefined stages: data understanding, exploration, preprocessing, model selection, hyperparameter tuning, evaluation, explanation, a global audit, and reporting.

The architecture uses Streamlit for the interface and LangGraph for a fixed sequential graph. Each stage reads a shared typed state and returns only its changes. Important choices are recorded with reasoning and an uncertainty indicator, so a reader can inspect how the result was obtained.

Most computation is ordinary Python and machine-learning libraries. Candidate models are compared using cross-validation for supervised tasks, or silhouette scores for unsupervised tasks. Optuna tunes the chosen family. SHAP and permutation importance provide complementary explanation diagnostics. Groq supplies narration and can assist target detection or a statistically inconclusive top-two model comparison; it does not plan the workflow or execute generated code.

The project includes dashboard results, a PDF report and a saved estimator. In the review, all 23 existing tests passed offline. The main limitations are preprocessing before splitting, shared artifact paths, incomplete raw-data model export and confidence indicators that are not calibrated probabilities. The next engineering priorities are to address those validity and reproducibility gaps rather than overstate autonomy.

Four short interview answers

Interview question	Accurate answer
Is orchestration dynamic?	No. The graph follows pipeline_steps in order. Conditional logic lives inside stages, not in runtime graph planning.
How do agents select tools?	They do not dynamically select arbitrary tools. Python rules call predefined functions; model-family candidates are coded, scores compare them, and a constrained LLM recommendation can choose between the top two.
How do you detect a wrong decision?	Inspect explicit outputs, scores, decision reasoning and arbitration provenance; check known-label tests and domain requirements. Low agreement and audit flags are indicators. Logs show what happened, not proof of correctness. Business intent and independent validation remain necessary.
What happens after an error?	Uncaught stage errors are logged and stop the graph; the UI clears stale results. Some local retries/fallbacks exist, but there is no checkpoint resume, autonomous graph repair or general code-repair mechanism.

Ten likely follow-up questions

1. Why use LangGraph for a linear pipeline? It provides explicit state merging and streamed node updates under a uniform graph contract. A simple loop could also implement this sequence; the repository does not establish a benchmark proving graph overhead is necessary. Evidence: supervisor_agent.py:102; streamlit_app.py:244.

2. Are all nine agents LLM agents? No. PP, HP, EV and ME have no LLM call; other stages use it for bounded decisions or narration. Evidence: their execute() methods in Q11 and the six-call inventory in Q23.

3. Can I trust a 95% confidence label? Not as a calibrated probability. It may be 1-p or a clamped heuristic; explain its exact source and limitations. Evidence: model_selection_agent.py:226; meta_evaluator_agent.py:66.

4. Does p>0.05 prove two models are equivalent? No. It is insufficient evidence against the test's null under assumptions; dependence and small samples further limit interpretation. Evidence: model_selection_agent.py:519 and Q20's exact-test probes.

5. Can the LLM really change the selected model? Yes, if a top-two recommendation parses to the runner-up. A mocked-score test verifies execute() returns that selection. Evidence: model_selection_agent.py:569; tests/test_model_selection_arbitration.py:146.

6. Is evaluation leakage-free? Estimator fitting uses the held-out split correctly, but full-data preprocessing leaks information. Evidence: preprocessing_agent.py:169; model_selection_agent.py:178; evaluation_agent.py:72.

7. Why combine SHAP and permutation importance? They measure different aspects of model behavior; rank agreement is a diagnostic cross-check, not causal verification. They currently use different row scopes. Evidence: explainability_agent.py:311 and :350; evaluation_agent.py:235.

8. Can the pickle predict a fresh raw CSV? Only after external reproduction of the required transformations and feature layout. The complete preprocessing/target mapping is not saved. Evidence: hyperparameter_agent.py:290; preprocessing_agent.py:157.

9. What if Groq is unavailable? The service retries where a client exists and then returns a deterministic fallback. Offline provenance can currently be misleading; no live failures were tested in this review. Evidence: llm_service.py:127; Q24 probe.

10. What do the passing tests prove? They prove 23 specific assertions about synthetic pipeline paths, parsing, skip behavior and label encoding under offline conditions. They do not prove unbiased metrics, multi-user safety or universal model quality. Evidence: tests/ inventory in Q28 and tmp/pdfs/test_results.json.

Honest statement of absent and partial capabilities

Absent: dynamic planning/tool use, generated-code execution/repair, persistent agent memory, graph checkpoint resume, runtime graph-level parallel agents, application authorization, prediction serving and automatic deployment. Partial: arbitrary-data robustness, independent evaluation, reusable raw-data inference, reliable LLM provenance, explanation consistency and confidence reporting. Implemented: a fixed graph, bounded rules/statistics/LLM assistance, local model tuning, result presentation and decision records. Proposed fixes remain proposals; no source changes were made for this guide.

3. Code evidence

• app/core/supervisor/supervisor_agent.py:40 and :102; app/core/agents/base_agent.py:26; app/core/state/emads_state.py:43.

• app/core/agents/model_selection_agent.py:146 and :487; hyperparameter_agent.py:105; evaluation_agent.py:42; explainability_agent.py:51; meta_evaluator_agent.py:66; reporting_agent.py:26. All share prefix app/core/agents/.

• app/services/llm_service.py:127; app/ui/streamlit_app.py:209 and :622; tests/test_model_selection_arbitration.py:146; tmp/pdfs/test_results.json.

4. Limitations or uncertainties

Presentation length depends on speaking speed; no recorded 90-second timing is claimed. Your CV, exact personal contributions, independent production results, deployment configuration and future interview questions are unknown. Adapt the template only to contributions you can personally substantiate. Refer to the glossary before using statistical terms, and avoid equating auditability with correctness.

5. Short interview answer

EMADS is a fixed, explainable ML workflow with bounded automated choices. Its strongest defensible story is traceability and modular execution, supported by specific tests. Its limitations are concrete engineering and statistical gaps that can be explained honestly and improved systematically.

Glossary

Term	Meaning
Agent (in EMADS)	A Python processing class with an execute(state) method; no general planning capability is implied.
Anomaly detection	Finding observations unusual relative to learned structure; not automatically equivalent to known fraud classification.
Arbitration	A secondary decision mechanism used when a predefined trigger is reached; here sometimes an LLM recommendation.
Artifact	A stored output such as a CSV, model pickle, plot, log or PDF.
Auditability	Ability to inspect what happened and the stated basis; it is not proof that the choice was correct.
Baseline	A reference configuration/score used to compare a changed model.
Calibration	Agreement between claimed probabilities and empirical event frequencies under a defined task/population.
Checkpoint	A saved execution-state snapshot that supports later resume; absent from this graph configuration.
Classification / regression	Predicting a category / a numeric quantity.
Clustering	Grouping similar observations without a supplied target label.
Contamination	Anomaly-estimator parameter describing an assumed outlier fraction or thresholding policy.
Cross-validation (CV)	Repeated fitting and validation using partitioned folds to compare or assess models.
CSV	Comma-separated values: a tabular text-file format.
Data leakage	Evaluation information influences fitting or model-choice decisions that should not have access to it.
DBSCAN	Density-Based Spatial Clustering of Applications with Noise; groups density-connected points and marks noise.
Decision record	EMADS AgentDecision: chosen action, reasoning, optional confidence indicator and timestamp.
Deterministic rule	A programmed mapping from inputs to behavior; it need not be statistically justified.
EDA	Exploratory data analysis: descriptive summaries and visual investigation before modeling.
Encoding	Representing categorical values numerically, such as one-hot columns or integer labels.
Ensemble / boosting	Combining models / sequentially adding models to improve an objective.

Term	Meaning
F1 / precision / recall	F1 combines precision and recall; precision concerns correctness of positive predictions, recall concerns recovery of actual positives.
Feature / target	An input predictor / the supervised output to be predicted.
Fold / stratification	One validation partition / partitioning that approximately preserves class proportions.
Holdout	A set excluded from estimator fitting and model selection; full independence also requires fitting preprocessing without it.
Hyperparameter	A model configuration chosen outside its ordinary fit, such as depth or regularization strength.
Imputation	Replacing missing feature values using an estimated or fixed value.
IQR	Interquartile range: 75th percentile minus 25th percentile.
KMeans	Centroid-based clustering minimizing within-cluster squared distances.
LangGraph / node / edge	A graph-execution library / a processing function / a permitted execution connection.
LLM / Groq	Large language model / the remote inference provider used by this repository.
LOF	Local Outlier Factor: local-density-based anomaly detection.
MAE / MSE / RMSE	Mean absolute error / mean squared error / square root of MSE.
Mock	A test substitute that returns controlled behavior instead of running a real dependency.
Nested CV	An outer evaluation loop around an inner model-selection/tuning loop; not configured here.
One-hot encoding	One binary feature per represented category; drop_first omits a reference category.
Optuna / TPE	Hyperparameter-optimization library / Tree-structured Parzen Estimator sampler using prior trials to guide suggestions.
p-value / null hypothesis	A tail-probability measure under a statistical reference model / the hypothesis tested; p is not a probability that a chosen model is correct.
Permutation importance	Mean performance drop when a feature is shuffled relative to a fixed evaluation sample.
Pickle	Python object serialization; loading untrusted files can execute unsafe behavior.
Pipeline / ColumnTransformer	Sklearn composition of transformations and estimator / column-specific transformation composition; proposed for full inference export.
R2	Coefficient of determination: squared-error improvement over a mean-only test baseline; can be negative.
Reducer	A rule that combines a new state update with an existing value.

Term	Meaning
ROC AUC	Area under the receiver operating characteristic curve; evaluates discrimination from prediction scores.
Seed	Initial value controlling pseudorandom choices; reproducibility also depends on software and execution details.
SHAP / Shapley value	SHapley Additive exPlanations / feature attribution based on average marginal contribution in a cooperative-game formulation.
Silhouette	Internal clustering measure comparing within-group and other-group distances; ranges from -1 to 1.
Spearman correlation	Correlation of ranks; measures monotonic ordering agreement, not correctness or causality.
StandardScaler	Sklearn transformer subtracting fitted means and dividing by fitted standard deviations.
State / TypedDict	Shared run-result dictionary / Python type annotation for dictionary fields, not complete runtime validation.
Wilcoxon signed-rank	Paired nonparametric test on signed ranks of differences; standard reference assumptions do not remove CV dependence.

Verification evidence and environment

Executed suite: 23 tests; 0 failures; 0 errors; 0 skipped; 56.163 seconds. This duration is not an application performance benchmark.

Offline LLM; dotenv loading suppressed; socket connections blocked; joblib threading backend; native thread pools limited to one; cwd isolated.

Package	Inspected installed version
numpy	2.4.6
pandas	3.0.3
scipy	1.18.0
scikit-learn	1.9.0
xgboost	3.3.0
lightgbm	4.7.0
optuna	4.9.0
shap	0.52.0
langgraph	1.2.8
streamlit	1.59.0
groq	1.5.0
reportlab	5.0.0
PyMuPDF	1.28.0

Individual existing tests (all passed)

Test identifier	Seconds (review conditions)
tests.test_explainability_unsupervised.ExplainabilityUnsupervisedSkipTests.test_dbscan_skips_shap	0.151
tests.test_explainability_unsupervised.ExplainabilityUnsupervisedSkipTests.test_isolation_forest_skips_shap	0.267
tests.test_explainability_unsupervised.ExplainabilityUnsupervisedSkipTests.test_kmeans_skips_shap	0.153
tests.test_explainability_unsupervised.ExplainabilityUnsupervisedSkipTests.test_local_outlier_factor_skips_shap	0.039
tests.test_explainability_unsupervised.ExplainabilityUnsupervisedSkipTests.test_skip_reasoning_says_skipped_by_design	0.022
tests.test_model_selection_arbitration.ModelSelectionArbitrationTests.test_llm_recommendation_overrides_higher_score	0.008
tests.test_model_selection_arbitration.ModelSelectionArbitrationTests.test_malformed_llm_response_falls_back_to_highest_score	0.005
tests.test_model_selection_arbitration.ModelSelectionArbitrationTests.test_significance_note_does_not_claim_recommendation_on_parse_failure	0.004
tests.test_model_selection_arbitration.ModelSelectionExecuteArbitrationTests.test_reasoning_does_not_falsely_claim_best_score_for_promoted_runner_up	0.055
tests.test_model_selection_arbitration.ModelSelectionExecuteArbitrationTests.test_selected_model_name_falls_back_deterministically_on_parse_failure	0.017
tests.test_model_selection_arbitration.ModelSelectionExecuteArbitrationTests.test_selected_model_name_follows_llm_recommendation_over_higher_score	0.016
tests.test_pipeline_edge_cases.PipelineEdgeCaseTests.test_categorical_target_uses_classification_metrics	16.518
tests.test_pipeline_edge_cases.PipelineEdgeCaseTests.test_empty_target_column_raises_clear_error	2.385
tests.test_pipeline_edge_cases.PipelineEdgeCaseTests.test_numeric_target_uses_regression_metrics	6.595
tests.test_target_arbitration_confidence.TargetArbitrationConfidenceTests.test_confidence_stays_within_documented_arbitration_band	0.003
tests.test_target_arbitration_confidence.TargetArbitrationConfidenceTests.test_low_and_high_unique_ratio_suggestions_produce_different_confidences	0.014
tests.test_target_arbitration_confidence.TargetArbitrationConfidenceTests.test_reasoning_never_asks_llm_to_self_rate_confidence	0.002

Test identifier	Seconds (review conditions)
tests.test_unsupervised_pipeline.UnsupervisedPipelineTests.test_anomaly_detection_produces_anomaly_metrics	8.514
tests.test_unsupervised_pipeline.UnsupervisedPipelineTests.test_clustering_produces_silhouette_metrics	4.574
tests.test_unsupervised_pipeline.UnsupervisedPipelineTests.test_unsupervised_requires_minimum_rows	0.049
tests.test_xgboost_label_encoding.HyperparameterAgentXGBoostNonContiguousClassesTests.test_tuning_and_final_pickle_survive_non_contiguous_classes	8.444
tests.test_xgboost_label_encoding.ModelSelectionXGBoostNonContiguousClassesTests.test_non_contiguous_classes_no_longer_fail	5.072
tests.test_xgboost_label_encoding.ModelSelectionXGBoostNonContiguousClassesTests.test_zero_indexed_contiguous_classes_still_work	3.255

Historical reports inspected for generic claims only

Text extraction and opening were verified. This is not a visual QA certificate for those historical files and does not attribute them to the inspected commit. No private run values are reproduced.

Repository-relative report	Pages / claim check
reports/EMADS_Report_20260809_135904.pdf	8 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260809_140002.pdf	6 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260809_140010.pdf	6 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260809_141802.pdf	12 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260809_142122.pdf	12 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260809_142934.pdf	7 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260810_175954.pdf	11 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260810_180846.pdf	13 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.
reports/EMADS_Report_20260810_182100.pdf	11 pages; statistics-grounded cover claim present; audit and statistical-verdict headings present.

Diagnostic probes

parser_negation: RandomForest

wilcoxon_exact_5: 0.0625

wilcoxon_exact_10: 0.001953125

wilcoxon_all_zero_default: 1.0

offline_target_fallback: {'column': 'rating', 'confidence': 0.73, 'claims_llm_arbitration': True, 'log_entries': 1}

xgb_refit: ValueError: Invalid classes inferred from unique values of y. Expected: [1 2 3], got [0 1 2]

zero_difference_comparison: {'p_value': '1.0', 'arbitration': True, 'llm_calls': 1}

Unresolved evidence

No supplied CV or personal-contribution record; no separate thesis report; no live provider/model-availability test; no production deployment/concurrency test; no independent benchmark or calibration study. Historical artifacts have no saved code-version manifest. These gaps are not filled by interpreting filenames or class names.

Source symbol index

This index is generated directly from the inspected Python syntax trees. It gives exact definition lines for classes/functions cited in the answers; body-line references in prose identify operations within them. Paths are repository-relative.

app/core/agents/base_agent.py

Class / function	Line
BaseAgent	26
BaseAgent.__init__	32
BaseAgent.decide	44
BaseAgent.log	58
BaseAgent.execute	63
BaseAgent.execute_logged	76

app/core/agents/data_understanding_agent.py

Class / function	Line
DataUnderstandingAgent	47
DataUnderstandingAgent.__init__	54
DataUnderstandingAgent.execute	58
DataUnderstandingAgent._load_dataset	130
DataUnderstandingAgent._profile_columns	149
DataUnderstandingAgent._resolve_target_column	166
DataUnderstandingAgent._arbitrate_target_column	224
DataUnderstandingAgent._skip_target_for_unsupervised	299
DataUnderstandingAgent._validate_row_count_for_unsupervised	320
DataUnderstandingAgent._infer_problem_type	330
DataUnderstandingAgent._validate_target_for_modeling	391
DataUnderstandingAgent._detect_quality_issues	410

app/core/agents/eda_agent.py

Class / function	Line
EDAAgent	24
EDAAgent.__init__	30
EDAAgent.execute	34
EDAAgent._build_fallback_eda_summary	137

app/core/agents/evaluation_agent.py

Class / function	Line
EvaluationAgent	33
EvaluationAgent.__init__	39
EvaluationAgent.execute	42
EvaluationAgent._evaluate_unsupervised	163
EvaluationAgent._cross_validate	217
EvaluationAgent._build_cv_splitter	226
EvaluationAgent._compute_permutation_importance	235
EvaluationAgent._build_permutation_decision	252
EvaluationAgent._compute_roc_auc	267
EvaluationAgent._build_stability_reasoning	281

app/core/agents/explainability_agent.py

Class / function	Line
ExplainabilityAgent	41
ExplainabilityAgent.__init__	47
ExplainabilityAgent.execute	51
ExplainabilityAgent._build_cross_check_note	238
ExplainabilityAgent._build_fallback_summary	253
ExplainabilityAgent._compute_feature_importance	298
ExplainabilityAgent._compute_shap	311
ExplainabilityAgent._agreement_score	350
ExplainabilityAgent._compute_local_explanations	371
ExplainabilityAgent._confidence_from_importance_margin	458
ExplainabilityAgent._top_feature	482
ExplainabilityAgent._build_llm_prompt	485

app/core/agents/hyperparameter_agent.py

Class / function	Line
XGBClassifierWithLabelEncoding	55
XGBClassifierWithLabelEncoding.fit	66
XGBClassifierWithLabelEncoding.predict	77
XGBClassifierWithLabelEncoding.classes_	82
HyperparameterAgent	96
HyperparameterAgent.__init__	102
HyperparameterAgent.execute	105
HyperparameterAgent._confidence_from_improvement	338
HyperparameterAgent._build_retry_decision	349
HyperparameterAgent._get_baseline_score	403
HyperparameterAgent._build_cv_splitter	410
HyperparameterAgent._widen_int_high	425
HyperparameterAgent._build_model_from_trial	440
HyperparameterAgent._build_model_from_params	535
HyperparameterAgent._build_reasoning	573

app/core/agents/meta_evaluator_agent.py

Class / function	Line
MetaEvaluatorAgent	57
MetaEvaluatorAgent.__init__	63
MetaEvaluatorAgent.execute	66
MetaEvaluatorAgent._metrics_component	133
MetaEvaluatorAgent._explanation_component	157
MetaEvaluatorAgent._arbitration_component	169
MetaEvaluatorAgent._build_recommendation	180
MetaEvaluatorAgent._find_target_decision	220

app/core/agents/model_selection_agent.py

Class / function	Line
XGBClassifierWithLabelEncoding	83
XGBClassifierWithLabelEncoding.fit	97
XGBClassifierWithLabelEncoding.predict	112
XGBClassifierWithLabelEncoding.classes_	117
ModelSelectionAgent	136
ModelSelectionAgent.__init__	142
ModelSelectionAgent.execute	146
ModelSelectionAgent._build_candidates	287
ModelSelectionAgent._score_candidates_supervised	322
ModelSelectionAgent._score_candidates_unsupervised	352
ModelSelectionAgent._build_cv_splitter	395
ModelSelectionAgent._build_reasoning	404
ModelSelectionAgent._confidence_from_margin	434
ModelSelectionAgent._confidence_from_unsupervised_margin	454
ModelSelectionAgent._compare_top_two	487
ModelSelectionAgent._parse_arbitration_recommendation	589
ModelSelectionAgent._narrow_margin_note	612
ModelSelectionAgent._build_significance_note	630

app/core/agents/preprocessing_agent.py

Class / function	Line
PreprocessingAgent	31
PreprocessingAgent.__init__	37
PreprocessingAgent.execute	40
PreprocessingAgent._select_columns_to_drop	209
PreprocessingAgent._confidence_from_drop_reasons	227
PreprocessingAgent._impute_missing_values	241
PreprocessingAgent._encode_categoricals	255

app/core/agents/reporting_agent.py

Class / function	Line
ReportingAgent	18
ReportingAgent.__init__	21
ReportingAgent.execute	26
ReportingAgent._build_dataset_summary_text	87

app/core/state/emads_state.py

Class / function	Line
AgentDecision	35
EMADSState	43
create_initial_state	103

app/core/supervisor/supervisor_agent.py

Class / function	Line
SupervisorAgent	32
SupervisorAgent.__init__	37
SupervisorAgent._wrap	65
SupervisorAgent._format_log	99
SupervisorAgent._build_workflow_graph	102
SupervisorAgent.run_pipeline	115

app/services/llm_service.py

Class / function	Line
LLMService	36
LLMService.__init__	53
LLMService._resolve_api_key	95
LLMService._resolve_api_key_source	115
LLMService._call_groq	127
LLMService.generate_eda_summary	202
LLMService.generate_summary	246

app/services/pdf_service.py

Class / function	Line
PDFService	43
PDFService.__init__	46
PDFService.generate_pdf_report	75
PDFService._build_title_page	115
PDFService._section_header	163
PDFService._build_section	169
PDFService._build_eda_section	172
PDFService._build_preprocessing_section	177
PDFService._build_model_section	197
PDFService._build_model_selection_verdict	226
PDFService._build_evaluation_section	284
PDFService._build_explainability_section	300
PDFService._build_local_explanations_section	306
PDFService._build_decisions_log	329
PDFService._build_arbitration_section	346
PDFService._build_confidence_audit_section	370
PDFService._confidence_color	400
PDFService._embed_images	407
PDFService._styled_table	415
PDFService._draw_header_footer	437
PDFService._safe_text	463

app/ui/streamlit_app.py

Class / function	Line
header	63
metric_card	72
section_title	76
_no_obvious_target_suggested	89
render_sidebar	116
run_pipeline_with_progress	209
_merge_stream_update	296
render_meta_evaluation_badge	307
_find_target_decision	322
render_overview_tab	333
render_eda_tab	358
render_preprocessing_tab	372
render_model_selection_verdict	393
render_model_tab	440
render_evaluation_tab	469
render_explainability_tab	526
render_arbitration_tab	586
render_decisions_tab	602
render_report_tab	622
render_empty_state	654
main	668

app/ui/theme.py

Class / function	Line
_status_colors	52
_build_css	61
inject_theme	231
status_from_thresholds	239
metric_card	250
badge	262
stepper_html	266
style_model_comparison	288
style_metrics_table	306

app/utils/file_utils.py

Class / function	Line
ensure_dir	16
output_path	23
prune_old_files	30

app/utils/logger.py

Class / function	Line
reset_session	34
_get_session_id	54
get_log_file_path	62
get_logger	68

app/utils/plot_utils.py

Class / function	Line
ensure_output_dir	24
save_current_figure	29
plot_correlation_heatmap	45
plot_confusion_matrix	54
plot_roc_curve	63
plot_missing_values	75
plot_distribution	87
plot_target_balance	94
plot_cluster_scatter	104
detect_outliers_iqr	121